

МИНОБРАЗОВАНИЯ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук
Кафедра цифровых технологий

**Прогнозирование субъективной оценки здоровья пациентов с
мочекаменной болезнью методами регрессионного моделирования**

Отчет по учебной практике
(научно-исследовательская работа)

02.03.01 Математика и компьютерные науки

Профиль «Распределенные системы и искусственный интеллект»

Зав. кафедрой  Кургалин С. Д. д.ф.-м.н., профессор 12.06.2026

Обучающийся  Борисов А. И.

Руководитель  Сирота Е. А. к.ф.-м.н., доцент

Оценка: „отлично“

Воронеж 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 Материалы и предварительная обработка данных	7
1.1 Характеристика исходной выборки и создание производных признаков	7
1.2 Обработка пропущенных значений методом MICE	8
1.3 Отбор предикторов и корреляционный анализ	9
1.3.1 Парные коэффициенты корреляции Пирсона	10
1.3.2 Мультиколлинеарность и её диагностика.....	12
1.4 Стандартизация признаков.....	14
2 Методологический аппарат исследования.....	16
2.1 Классическая линейная регрессия и метод наименьших квадратов.....	16
2.2 Регуляризация: гребневая регрессия и Elastic Net	17
2.3 Полиномиальное расширение пространства предикторов	18
2.4 Бинарная логистическая регрессия и критерии качества бинарной классификации.....	19
3 Вычислительные эксперименты	20
3.1 Линейные модели.....	20
3.1.1 Модель OLS с контролем мультиколлинеарности.....	20
3.1.2 Гребневая регрессия на полном наборе предикторов	21
3.1.3 Сравнительный анализ линейных спецификаций	22
3.2 Полиномиальная регрессия с регуляризацией	23
3.2.1 Генерация мономов второй степени и селекция Elastic Net	23
3.2.2 Обучение финальной Ridge-модели на редуцированном пространстве	23
3.2.3 Оценка качества и интерпретация нелинейных эффектов.....	24
3.3 Бинарная логистическая модель.....	26
3.3.1 Бинаризация отклика и оценивание параметров	26
3.3.2 Кросс-валидация и анализ прогностической способности	28
4 Программная архитектура и организация вычислительного эксперимента	29
4.1 Обоснование выбора технологического стека	29
4.2 Структура программного модуля и организация вычислительного конвейера.....	29

4.3 Обеспечение воспроизводимости.....	31
4.4 Краткая характеристика программного кода.....	32
5 Обсуждение результатов.....	33
5.1 Сопоставительный анализ прогностической точности.....	33
5.2 Интерпретация значимых факторов.....	34
5.3 Ограничения и направления дальнейших исследований.....	34
ЗАКЛЮЧЕНИЕ	36
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	39
ПРИЛОЖЕНИЕ А	40

ВВЕДЕНИЕ

Разработка методов количественного прогнозирования на основе многомерных эмпирических данных относится к числу фундаментальных задач прикладной статистики и машинного обучения. Исходный материал для настоящего исследования получен в результате анкетирования 800 пациентов урологического профиля Воронежской области, проведённого совместно с Воронежским государственным медицинским университетом имени Бурденко. Сложность задачи обусловлена тем, что реальные объектные системы редко допускают адекватное описание в терминах простых линейных соотношений, а размерность признакового пространства часто превышает объём доступной выборки, порождая риск переобучения и нестабильности получаемых решений [5]. В этих условиях выбор математического аппарата перестаёт быть технической формальностью и превращается в самостоятельную исследовательскую проблему.

Настоящая работа посвящена именно такому сопоставительному анализу. Предметной областью выступает прогнозирование субъективной оценки здоровья пациентов с мочекаменной болезнью, интегрального показателя, который агрегирует как объективное соматическое состояние, так и индивидуальное психоэмоциональное восприятие собственного благополучия. Мочекаменная болезнь характеризуется высокой распространённостью, склонностью к хроническому рецидивирующему течению и значительным влиянием на качество жизни, что делает задачу количественной оценки субъективного здоровья клинически осмысленной. Основная тяжесть исследования при этом смещена в область вычислительных методов: центральный вопрос заключается не столько в медицинской интерпретации конкретных предикторов, сколько в том, какая математическая стратегия позволяет наиболее пол-

но извлечь информацию из ограниченного набора клинико-анамнестических данных.

Целью работы является сравнительное исследование трёх методологически различных подходов к регрессионному моделированию субъективной оценки здоровья при мочекаменной болезни, а именно классических линейных моделей, регуляризованной полиномиальной регрессии с автоматическим отбором мономов и бинарной логистической регрессии. Итогом исследования выступает не только совокупность обученных моделей с измеренными метриками качества, но и обоснованный вывод о сравнительной эффективности указанных подходов, условиях их применимости и достигаемом компромиссе между точностью аппроксимации, интерпретируемостью и устойчивостью к переобучению.

Для достижения сформулированной цели в работе последовательно решаются следующие задачи. Выполняется предобработка исходного массива анкетных данных, включающая восстановление пропущенных значений методом множественной импутации цепными уравнениями (MICE) и конструирование производных признаков, отражающих наследственную отягощённость, продолжительность заболевания и коморбидность. Проводится корреляционный анализ и диагностика мультиколлинеарности набора из 18 предикторов, сформированного совместно с врачами экспертами. Реализуются и оцениваются две линейные спецификации: модель, построенная методом наименьших квадратов с исключением одного избыточного признака, и гребневая регрессия на полном наборе переменных. Пространство признаков расширяется полиномами второй степени, после чего применяется двухэтапная процедура регуляризации, состоящая из селекции мономов методом Elastic Net и последующей гребневой регрессии на редуцированном наборе из 75 мономов. Строится бинарная логистическая регрессия, классифицирующая пациентов по уровню самооценки относительно медианного значения. Все вычислительные процедуры реализуются в виде единого программного

модуля на языке Python с документированным конвейером обработки данных. Проводится сравнительная оценка качества полученных моделей методом пятикратной кросс-валидации с использованием набора метрик, адекватных природе каждой спецификации.

Научная новизна исследования определяется тремя обстоятельствами. В работе впервые для задачи прогнозирования субъективной оценки здоровья при мочекаменной болезни применена и эмпирически обоснована комбинированная стратегия регуляризации, которая объединяет полиномиальное расширение признакового пространства, автоматический отбор мономов с помощью Elastic Net и финальную гребневую регрессию. Показано, что данный подход позволяет более чем вдвое повысить долю объяснённой дисперсии по сравнению с линейным базовым уровнем, что служит прямым эмпирическим свидетельством существенной роли нелинейных взаимодействий и квадратичных эффектов в формировании субъективной оценки здоровья. Кроме того, в явном виде сопоставлены три методологии (линейная, нелинейная регуляризованная и бинарная классификационная), что позволяет сформулировать обоснованные рекомендации относительно выбора математического аппарата в зависимости от прикладной цели, будь то количественный прогноз или скрининговая стратификация. Наконец, разработанный программный модуль оформлен как воспроизводимый вычислительный эксперимент с фиксацией параметров импутации, регуляризации и кросс-валидации, что обеспечивает верифицируемость полученных результатов и возможность их независимой проверки.

Практическая значимость работы заключается в создании инструментария, который может быть интегрирован в системы поддержки принятия врачебных решений для стратификации пациентов с мочекаменной болезнью по риску низкой субъективной оценки здоровья и, как следствие, низкой приверженности терапии. С математической точки зрения апробированная в работе схема расширения признакового пространства полиномами с после-

дующей двухэтапной регуляризацией носит универсальный характер и может быть перенесена на широкий класс задач анализа многомерных эмпирических данных в медицине, социологии и смежных областях.

1 Материалы и предварительная обработка данных

1.1 Характеристика исходной выборки и создание производных признаков

Исходный массив данных сформирован по результатам анкетирования пациентов урологического профиля, проходивших обследование и лечение в медицинских учреждениях города Воронежа. Первичная база содержала 800 записей и 115 показателей, отражающих социально-демографические характеристики, анамнез заболевания, наследственность, сопутствующую патологию, особенности образа жизни и клинические параметры. Для целей настоящего исследования из общей совокупности выделена основная группа, которую составили 400 пациентов с верифицированным диагнозом «Мочекаменная болезнь» (код группы 1). Остальные записи, соответствующие контрольной группе, в дальнейший анализ не включались.

На этапе подготовки выполнено конструирование дополнительных признаков, призванных более полно отразить наследственную предрасположенность, длительность течения заболевания и коморбидный фон. Шесть дихотомических переменных, кодирующих наличие мочекаменной болезни у различных категорий родственников (дедушка или бабушка, тётя или дядя, двоюродные братья или сёстры, родители, сиблинги, дети), преобразованы в числовые индикаторы $h_i \in \{0, 1\}$. На их основе сформирован интегральный индекс наследственной отягощённости, вычисляемый как сумма значений по всем шести линиям:

$$I_{\text{насл}} = \sum_{i=1}^6 h_i$$

Индекс принимает целочисленные значения от 0 до 6 и характеризует общую наследственную нагрузку. Длительность заболевания определена как разность между текущим возрастом пациента и возрастом постановки диа-

гноза; отрицательные значения, возникавшие вследствие ошибок ввода, заменялись пропусками для последующего восстановления. Интегральный показатель сопутствующих болезней построен суммированием трёх бинарных индикаторов, отражающих наличие хронических заболеваний сердечно-сосудистой системы, хронических заболеваний эндокринной системы и сахарного диабета, и принимает значения от 0 до 3.

1.2 Обработка пропущенных значений методом MICE

Предварительный анализ выявил значительную долю отсутствующих значений в ряде переменных. Для двух признаков, а именно «Длительность сахарного диабета» и «Длительность курения», доля пропусков в подвыборке пациентов с известным значением целевой переменной составила 84,7 % и 73,1 % соответственно. Следуя общепринятым рекомендациям, переменные с уровнем пропусков выше 50 % были исключены из дальнейшего анализа, поскольку их восстановление методами импутации сопряжено с высоким риском внесения систематических ошибок. Оставшиеся пропуски, доли которых варьировали от 0,5 % до 41,7 % для различных признаков, восстановлены методом множественной импутации цепными уравнениями (Multiple Imputation by Chained Equations, MICE) [1].

Метод MICE реализует итеративную процедуру, в которой каждая переменная с пропусками поочерёдно моделируется как зависимая, а все остальные переменные выступают в роли предикторов. На каждом шаге для заполнения пропусков в очередной переменной строится регрессионная модель: линейная регрессия для непрерывных признаков и логистическая для бинарных. Предсказанные значения подставляются на место пропусков, после чего процедура повторяется для следующей переменной. Процесс продолжается до достижения сходимости; в настоящей работе выполнено 10 итераций. Принципиальное преимущество множественной импутации по сравнению с однократным заполнением средними или медианами состоит в

том, что она сохраняет ковариационную структуру данных и даёт несмещённые оценки дисперсии.

Из исходной группы 400 пациентов с мочекаменной болезнью 9 записей имели пропуск в целевой переменной «Оценка состояния здоровья» и были исключены из рассмотрения. Оставшаяся выборка объёмом 391 наблюдение подверглась процедуре MICE. Среди предикторов с ненулевой долей пропусков, помимо двух исключённых, наибольшие значения зафиксированы для переменных «Возраст начала приёма алкоголя» (41,7 %), «Стаж работы на последнем месте работы» (15,9 %), «Оценка условий труда» (11,3 %), «Физическая нагрузка на работе» (9,2 %), «Частота вызова скорой помощи» (8,4 %) и «Регулярный приём препаратов» (5,6 %). Доли пропусков остальных признаков не превысили 4,1 %. После выполнения 10 итераций MICE остаточные пропуски полностью отсутствовали, что подтверждено программной проверкой. Применение метода MICE в настоящей работе выполнено в соответствии с подходом, подробно изложенным в [1].

1.3 Отбор предикторов и корреляционный анализ

Формирование набора предикторов для моделирования самооценки здоровья осуществлялось в два этапа. На первом этапе совместно с врачами-экспертами выделены три группы факторов: базовые медико-биологические (23 признака), базовые социальные (8 признаков) и специфичные для данной целевой переменной (18 признаков). Итоговый список из 18 предикторов, согласованный с экспертами, включает следующие переменные: материальное положение семьи, личное материальное положение, отсутствие хронических заболеваний, семейное положение, оценка условий труда, жилищно-бытовые условия, частота отдыха, образование, возраст начала приёма алкоголя, возраст постановки диагноза, стаж работы на последнем месте работы, интегральный индекс сопутствующих болезней, боль в пояснице, возраст, сахарный диабет, соблюдение диеты, проведение профилактики мочекаменной бо-

лезни и оценка виртуальных консультаций. На втором этапе для данного набора выполнен корреляционный анализ по Пирсону как с целевой переменной, так и со всеми 120 исходными признаками набора данных, что позволило количественно оценить силу и направление линейных связей. По результатам этого анализа были выявлены дополнительные переменные, продемонстрировавшие устойчивые корреляции с целевой переменной, однако не вошедшие в первоначальный экспертный перечень, — в частности, «Возраст начала трудовой деятельности», «Снижение работоспособности» и «ХЗ сердечно-сосудистой системы». Тем не менее окончательный набор предикторов был сохранён в объёме 18 переменных, согласованных с экспертами, что мотивировано необходимостью сохранения клинической интерпретируемости модели и предотвращения возможного переобучения, которое могло бы возникнуть при расширении признакового пространства на основе апостериорного анализа корреляций в той же выборке. Выявленные связи со 120 признаками рассматриваются как эмпирическое обоснование для дальнейших исследований на независимых данных.

1.3.1 Парные коэффициенты корреляции Пирсона

Коэффициент корреляции Пирсона между переменной X и целевой переменной Y определяется выражением:

$$r = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2} \sqrt{\sum_{i=1}^n (Y_i - \bar{Y})^2}}, \quad (1)$$

где $\bar{X}$ и $\bar{Y}$ обозначают выборочные средние, а n — число наблюдений. Коэффициент принимает значения в диапазоне от минус единицы до единицы; значения, близкие к единице, указывают на прямую линейную зависи-

мость, близкие к минус единице — на обратную, близкие к нулю — на отсутствие линейной связи.

Корреляционный анализ 18 предикторов с целевой переменной «Оценка состояния здоровья» выявил следующую структуру связей. Вычисленные с помощью формулы (1) значения r показали, что наиболее сильная положительная корреляция зафиксирована для признака «Материальное положение семьи» ($r = 0,381$). Далее по убыванию следуют: «Материальное положение» ($r = 0,329$), «ХЗ не имею» ($r = 0,247$), «Семейное положение» ($r = 0,216$), «Жилищно-бытовые условия» ($r = 0,191$), «Образование» ($r = 0,187$), «Оценка условий труда» ($r = 0,186$), «Частота отдыха» ($r = 0,185$), «Возраст начала приёма алкоголя» ($r = 0,148$), «Проводилась профилактика МКБ» ($r = 0,148$), «Оценка виртуальных консультаций» ($r = 0,147$), «Соблюдение диеты» ($r = 0,142$). Среди отрицательных корреляций доминируют «Стаж работы на последнем месте работы» ($r = -0,378$), «Боль в пояснице» ($r = -0,366$), «Сопутствующие болезни» ($r = -0,365$), «Сахарный диабет» ($r = -0,350$), «Возраст постановки диагноза» ($r = -0,350$), «Возраст» ($r = -0,345$).

Анализ полного набора из 120 исходных признаков позволил выявить дополнительные, менее очевидные взаимосвязи. Так, «Возраст начала трудовой деятельности» продемонстрировал положительную корреляцию ($r = 0,283$), а «Снижение работоспособности» — выраженную отрицательную ($r = -0,325$). Признак «ХЗ сердечно-сосудистой системы» также обнаружил заметную отрицательную связь ($r = -0,304$). Эти результаты подтверждают, что субъективная оценка здоровья формируется под комплексным воздействием социально-экономических, клинических и поведенческих факторов, причём негативные клинические проявления оказывают на неё более сильное влияние, чем позитивные социальные параметры.

1.3.2 Мультиколлинеарность и её диагностика

Парные корреляции, рассмотренные выше, характеризуют связь каждого отдельного фактора с целевой переменной, но не отражают взаимозависимостей между самими предикторами. Наличие сильных линейных связей внутри набора признаков, называемое мультиколлинеарностью, способно дестабилизировать оценки коэффициентов регрессии, привести к неоправданно большим стандартным ошибкам и, как следствие, к неверным выводам о значимости переменных. По этой причине следующим этапом исследования стал анализ корреляционной матрицы предикторов и расчёт фактора инфляции дисперсии (Variance Inflation Factor, VIF).

На рисунке 1 представлена корреляционная матрица 18 предикторов, вычисленная на стандартизированных данных. Матрица отображает только верхнюю треугольную часть, поскольку является симметричной. Цветовая шкала варьируется от красного тона, соответствующего отрицательной корреляции, до синего, соответствующего положительной.

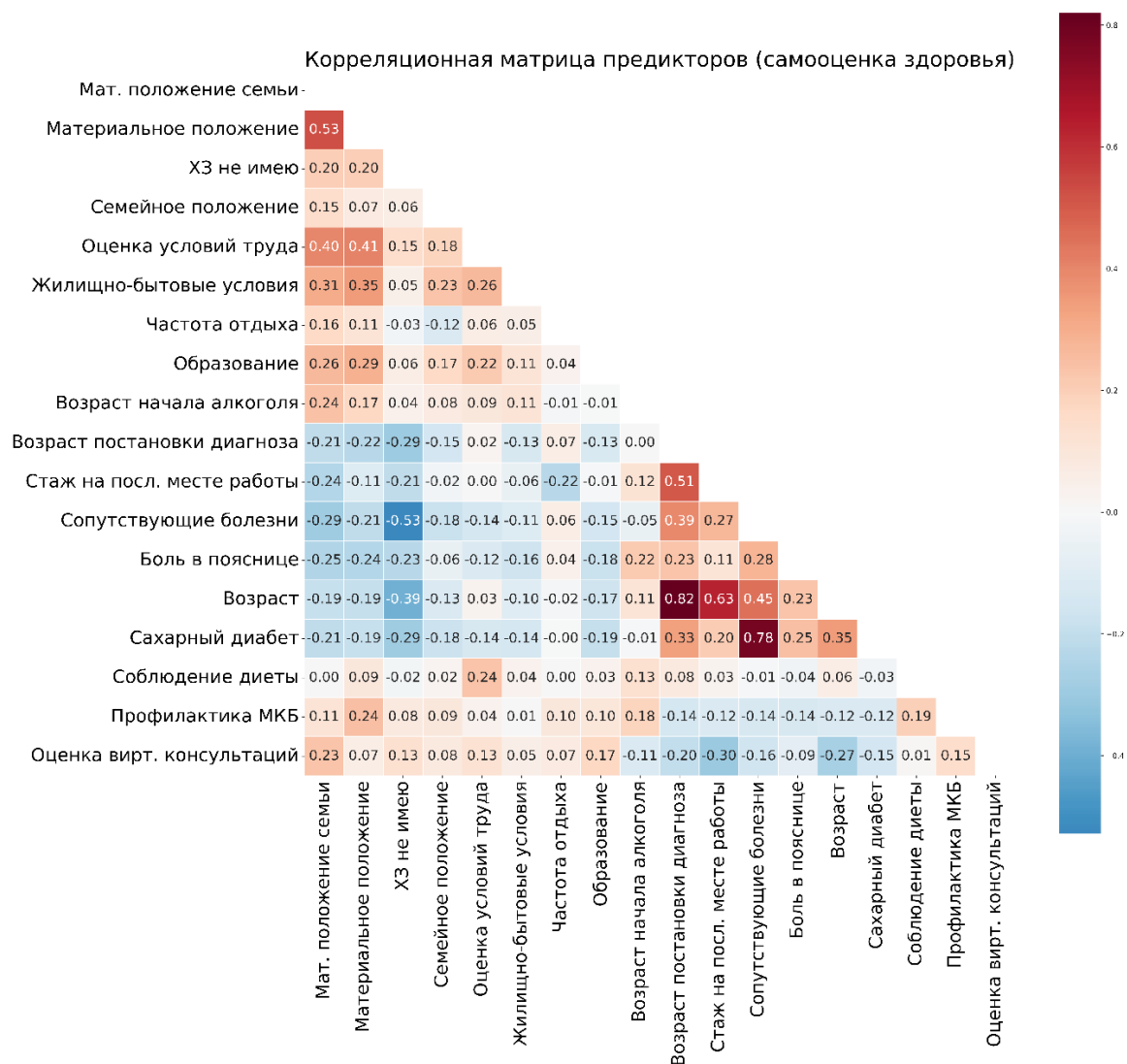


Рисунок 1 – Корреляционная матрица 18 исходных предикторов, отобранных для прогнозирования самооценки здоровья

Матрица подтверждает наличие заметных линейных связей между отдельными признаками. Коэффициент корреляции между возрастом и возрастом постановки диагноза достигает 0,82; высокая положительная связь ($r = 0,78$) наблюдается также между показателем сопутствующих болезней и сахарным диабетом. Умеренная корреляция ($r \approx 0,68$) зафиксирована между переменными «Материальное положение семьи» и «Материальное положение». Подобные зависимости обосновывают необходимость контроля мультиколлинеарности при построении регрессионных моделей.

Количественной мерой мультиколлинеарности служит фактор инфляции дисперсии, определяемый для каждого предиктора с индексом j выражением:

$$VIF_j = \frac{1}{1 - R_j^2}, \quad (2)$$

где R_j^2 представляет собой коэффициент детерминации вспомогательной регрессии, в которой j -й предиктор выступает в роли зависимой переменной, а все остальные предикторы — в роли независимых. Значение VIF, равное единице, соответствует полному отсутствию корреляции данного предиктора с остальными; значения, превышающие 5 (а по более строгим критериям — 10), свидетельствуют о проблемной мультиколлинеарности.

Значения фактора инфляции дисперсии, найденные по формуле (2), не превышают критического порога. Максимальное значение зафиксировано для переменной «Возраст» ($VIF = 4,43$), за которой следуют «Сопутствующие болезни» (3,73), «Возраст постановки диагноза» (3,37), «Сахарный диабет» (2,83) и «Стаж работы на последнем месте работы» (1,99). Для остальных признаков значения VIF не превосходят 1,93. Таким образом, формальные критерии не фиксируют критической мультиколлинеарности. Тем не менее для повышения устойчивости моделей в дальнейшем применяются два взаимодополняющих подхода: удаление одного из пары сильно коррелирующих признаков при построении классической линейной регрессии и регуляризация (Ridge, Elastic Net), естественным образом компенсирующая эффекты мультиколлинеарности.

1.4 Стандартизация признаков

Все непрерывные предикторы, используемые в моделях, были подвергнуты стандартизации, определяемой следующим образом:

$$X' = \frac{X - \mu}{\sigma}, \quad (3)$$

где μ обозначает выборочное среднее признака, а σ — его выборочное стандартное отклонение. После преобразования каждый признак приобретает нулевое среднее и единичную дисперсию. Стандартизация выполнена до этапа обучения моделей (за исключением случаев, когда использовались исходные данные, например, при расчёте полных корреляций). Применение данного преобразования позволяет сопоставлять коэффициенты регрессии, выраженные в единицах стандартного отклонения, и делает процедуры регуляризации корректными, поскольку все переменные вносят вклад в штрафное слагаемое в одном масштабе.

2 Методологический аппарат исследования

2.1 Классическая линейная регрессия и метод наименьших квадратов

Пусть имеется набор из n наблюдений и p предикторов. Линейная регрессионная модель постулирует, что целевая переменная Y представима в виде линейной комбинации предикторов с аддитивным случайным возмущением:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon, \quad (4)$$

где β_0 — свободный член, β_j ($j = 1, \dots, p$) — неизвестные коэффициенты, подлежащие оцениванию, а ε — случайная ошибка с нулевым математическим ожиданием и дисперсией σ^2 . Исторически наиболее распространённым методом оценивания коэффициентов является метод наименьших квадратов (Ordinary Least Squares, OLS), который отыскивает вектор β , минимизирующий сумму квадратов невязок:

$$\hat{\beta} = \arg \min_{\beta} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2. \quad (5)$$

При выполнении классических предположений (отсутствие мультиколлинеарности, гомоскедастичность, некоррелированность ошибок) оценки $\hat{\beta}$ являются несмещёнными и обладают минимальной дисперсией в классе линейных несмещённых оценок. Однако на практике перечисленные условия редко выполняются в полном объёме. Особую проблему представляет мультиколлинеарность, количественно диагностируемая фактором инфляции дисперсии, рассмотренным в подразделе 1.3.2. При её наличии дисперсии оце-

нок $\hat{\beta}_j$ резко возрастают, а сами коэффициенты становятся чувствительными к малым изменениям исходных данных.

2.2 Регуляризация: гребневая регрессия и Elastic Net

Для линейной модели (4) классический МНК даёт оценки (5), однако при мультиколлинеарности они становятся нестабильными. Для смягчения последствий мультиколлинеарности и предотвращения переобучения применяют регуляризованные методы, модифицирующие функцию потерь путём добавления штрафного слагаемого. Все предикторы предварительно были стандартизованы в соответствии с выражением (3). Гребневая регрессия (Ridge regression) [2] минимизирует сумму квадратов ошибок с квадратичным штрафом на величину коэффициентов:

$$\hat{\beta}_{\text{Ridge}} = \arg \min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\}, \quad (6)$$

где $\lambda \geq 0$ — параметр регуляризации, контролирующий степень сжатия коэффициентов. При $\lambda = 0$ модель (6) сводится к классическому OLS (5); с ростом λ коэффициенты приближаются к нулю, но не обращаются в нуль в точности. Гребневая регрессия ценой внесения небольшого смещения позволяет радикально уменьшить дисперсию оценок, повышая обобщающую способность модели.

Метод Elastic Net [3] объединяет L1- и L2-штрафы, что даёт возможность одновременно сжимать коэффициенты и автоматически отбирать значимые переменные, обнуляя часть из них. Целевая функция имеет вид:

$$\hat{\beta}_{\text{EN}} = \arg \min_{\beta} \left\{ \frac{1}{2n} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \right. \\ \left. + \lambda \sum_{j=1}^p \left(\frac{1-\alpha}{2} \beta_j^2 + \alpha |\beta_j| \right) \right\}, \quad (7)$$

где параметр $\alpha \in [0,1]$ задаёт баланс между гребневым ($\alpha = 0$) и лассо-штрафом ($\alpha = 1$). Сочетание двух типов регуляризации особенно полезно в ситуациях, когда число предикторов велико относительно объёма выборки, а часть переменных коррелирована между собой. Оптимальные значения λ и α определяются путём минимизации ошибки на отложенных фолдах кросс-валидации.

2.3 Полиномиальное расширение пространства предикторов

Линейная регрессия улавливает лишь линейные зависимости между предикторами и целевой переменной. Для учёта нелинейных эффектов, а также взаимодействий между переменными применяется полиномиальное расширение. Пусть исходный набор содержит p признаков. Построим все мономы степени не выше d : для $d = 2$ это линейные члены $X_1, \dots, X_p$, квадраты $X_1^2, \dots, X_p^2$ попарные произведения $X_i X_j$ ($i \neq j$). Общее число сгенерированных мономов составляет $p(p + 3) / 2$. В настоящей работе $p = 18$, что даёт 189 признаков.

Переход к полиномиальному пространству существенно увеличивает размерность задачи, делая её заведомо неблагоприятной для обычного OLS. Именно поэтому полиномиальное расширение комбинируют с регуляризацией: сначала с помощью Elastic Net (7) отбираются мономы, обладающие наибольшей предсказательной силой, а затем на редуцированном наборе

строится финальная гребневая модель (6), гарантирующая стабильность оценок.

2.4 Бинарная логистическая регрессия и критерии качества бинарной классификации

Задача бинарной классификации состоит в отнесении каждого наблюдения к одному из двух классов по значениям предикторов. В данной работе в качестве классификационной модели используется логистическая регрессия, которая оценивает апостериорную вероятность принадлежности к классу 1:

$$P(Y = 1 | X) = \frac{1}{1 + \exp(-a)}, \quad (8)$$

где $a = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$ — линейный предиктор. Параметры модели β находятся методом максимального правдоподобия путём численной оптимизации логарифмической функции правдоподобия.

Качество бинарного классификатора оценивается комплексом метрик. Доля правильных ответов (Ассигасу) характеризует общую точность, однако может вводить в заблуждение при несбалансированных классах. Площадь под ROC-кривой (ROC AUC) показывает способность модели разделять два класса независимо от выбранного порога классификации: значение 0,5 соответствует случайному гаданию, 1,0 — идеальному разделению. Балл Бриера (Brier Score) представляет собой среднеквадратичную ошибку предсказанных вероятностей и принимает значения от 0 (идеальный прогноз) до 1. Псевдо-R² МакФаддена (McFadden's pseudo-R²) определяет относительное улучшение логарифмического правдоподобия по сравнению с моделью, содержащей только константу; значения в диапазоне 0,2–0,4 расцениваются как свидетельство удовлетворительного качества.

3 Вычислительные эксперименты

Данный раздел содержит описание процесса построения и сравнительной оценки трёх классов прогностических моделей, предназначенных для количественного описания субъективной оценки здоровья пациентов с мочекаменной болезнью. Все вычисления выполнены на стандартизированной выборке из 391 наблюдения и 18 исходных признаков, очищенной от пропусков методом MICE.

3.1 Линейные модели

3.1.1 Модель OLS с контролем мультиколлинеарности

Первой из построенных линейных спецификаций является модель, оценённая методом наименьших квадратов с предварительным удалением переменной «Материальное положение». Удаление данного признака, коррелирующего с переменной «Материальное положение семьи» на уровне 0,68, позволило дополнительно снизить значения фактора инфляции дисперсии, хотя формальные критерии мультиколлинеарности и без того не были нарушены (максимальный VIF после удаления составил 4,42).

Уравнение модели в стандартизированных переменных записывается следующим образом:

$$\begin{aligned}\hat{Y} = & 0,3905 + 0,0233 \times Z_1 + 0,0097 \times Z_2 + 0,0340 \times Z_3 + \\ & + 0,0005 \times Z_4 - 0,0018 \times Z_5 + 0,0383 \times Z_6 + 0,0111 \times Z_7 + \\ & + 0,0453 \times Z_8 - 0,0301 \times Z_9 - 0,0605 \times Z_{10} - 0,0065 \times Z_{11} - \\ & - 0,0688 \times Z_{12} + 0,0147 \times Z_{13} - 0,0286 \times Z_{14} + 0,0287 \times Z_{15} - \\ & - 0,0125 \times Z_{16} - 0,0040 \times Z_{17}.\end{aligned}\tag{9}$$

В формуле (9) приняты следующие обозначения: Z_1 — материальное положение семьи, Z_2 — отсутствие хронических заболеваний, Z_3 — семейное

положение, Z_4 — оценка условий труда, Z_5 — жилищно-бытовые условия, Z_6 — частота отдыха, Z_7 — образование, Z_8 — возраст начала приёма алкоголя, Z_9 — возраст постановки диагноза, Z_{10} — стаж работы на последнем месте, Z_{11} — интегральный индекс сопутствующих болезней, Z_{12} — боль в пояснице, Z_{13} — возраст, Z_{14} — сахарный диабет, Z_{15} — соблюдение диеты, Z_{16} — проведение профилактики МКБ, Z_{17} — оценка виртуальных консультаций. Все переменные стандартизованы.

Коэффициент детерминации на обучающей выборке составил $R^2 = 0,427$, скорректированный коэффициент детерминации $R^2_{adj} = 0,401$. Оценки, полученные методом пятикратной кросс-валидации, оказались несколько ниже: $R^2_{CV} = 0,354$ со стандартным отклонением $0,0480$, $RMSE_{CV} = 0,187 \pm 0,014$, $MAE_{CV} = 0,148 \pm 0,011$. Различие между R^2 на обучении и на кросс-валидации невелико, что свидетельствует об отсутствии выраженного переобучения, однако абсолютное значение объяснённой дисперсии остаётся умеренным: модель улавливает лишь около трети вариации целевого показателя.

3.1.2 Гребневая регрессия на полном наборе предикторов

Вторая линейная спецификация представлена гребневой регрессией (Ridge), применённой ко всем 18 стандартизованным признакам без удаления каких-либо переменных. Оптимальное значение параметра регуляризации α определено методом пятикратной кросс-валидации (RidgeCV) по логарифмической сетке из 50 значений в диапазоне от 10^{-3} до 10^3 с использованием отрицательной среднеквадратичной ошибки в качестве критерия. Найденное значение составило $\alpha = 19,3070$.

Стандартизованные коэффициенты гребневой модели, упорядоченные по убыванию модуля, составили: возраст начала приёма алкоголя $+0,0411$; частота отдыха $+0,0356$; семейное положение $+0,0338$; соблюдение диеты $+0,0270$; материальное положение $+0,0230$; материальное положение

семьи +0,0172; возраст +0,0082; образование +0,0076; отсутствие хронических заболеваний +0,0071; оценка виртуальных консультаций –0,0014; оценка условий труда –0,0031; жилищно-бытовые условия –0,0042; сопутствующие болезни –0,0098; профилактика МКБ –0,0137; возраст постановки диагноза –0,0236; сахарный диабет –0,0262; стаж работы на последнем месте –0,0563; боль в пояснице –0,0629.

Качество данной модели по сравнению с OLS-спецификацией изменилось незначительно. На обучающей выборке получены значения $R^2 = 0,432$, $RMSE = 0,179$, $MAE = 0,141$. Кросс-валидационные метрики составили $R^2_{CV} = 0,361$, $RMSE_{CV} = 0,186$, $MAE_{CV} = 0,146$.

3.1.3 Сравнительный анализ линейных спецификаций

Прямое сопоставление двух линейных подходов представлено в таблице 1.

Таблица 1 – Сравнение линейных моделей самооценки здоровья (кросс-валидация)

Модель	R^2 (CV)	RMSE (CV)	MAE (CV)
OLS (удалён признак)	0,354	0,187	0,148
Ridge (все признаки)	0,361	0,186	0,146

Гребневая регрессия демонстрирует маргинальное превосходство по всем трём метрикам, однако прирост качества не является существенным. Обе линейные спецификации объясняют примерно 35–36 % дисперсии на кросс-валидации, что интерпретируется как указание на наличие значительной нелинейной составляющей в структуре данных, не улавливаемой линейными методами.

3.2 Полиномиальная регрессия с регуляризацией

3.2.1 Генерация мономов второй степени и селекция Elastic Net

Для учёта нелинейных эффектов и взаимодействий между предикторами выполнено расширение пространства признаков полиномами второй степени. С помощью класса `PolynomialFeatures` библиотеки `scikit-learn` (параметры `degree=2`, `include_bias=False`) из 18 исходных стандартизированных переменных сгенерировано 189 мономов, включающих линейные члены, квадраты и попарные произведения.

Работа со 189 признаками при 391 наблюдении создаёт риск переобучения, поэтому на следующем шаге применён метод Elastic Net. Целевая функция данного метода задаётся выражением (7), предложенным в [3].

Оптимальные параметры λ и α подобраны пятикратной кросс-валидацией на сетке значений $\alpha \in \{0,1; 0,5; 0,7; 0,9; 0,95; 1,0\}$. В результате получено $\lambda_{opt} = 0,0073$, $\alpha_{opt} = 0,10$. Высокая доля L2-штрафа (90 %) обеспечивает стабильность оценок, тогда как 10-процентная доля L1-штрафа позволяет обнулить часть коэффициентов. С помощью селектора `SelectFromModel` из 189 мономов отобрано 75 наиболее информативных.

3.2.2 Обучение финальной Ridge-модели на редуцированном пространстве

На отобранных 75 мономах построена финальная гребневая регрессия. Параметр регуляризации α_{Ridge} определён независимой пятикратной кросс-валидацией ($Ridge_{CV}$) по логарифмической сетке и составил 2,6827. Целевая функция записывается как

$$\min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^{75} \beta_j P_{ij} \right)^2 + \alpha_{Ridge} \sum_{j=1}^{75} \beta_j^2 \right\}, \quad (10)$$

где P_{ij} — значения стандартизированных мономов.

Качество модели (10) на обучающей выборке: $R^2 = 0,7986$, скорректированный коэффициент детерминации $R^2_{adj} = 0,7507$, $RMSE = 0,1069$, $MAE = 0,0834$. Кросс-валидационные метрики составили $R^2_{CV} = 0,7116 \pm 0,0189$, $RMSE_{CV} = 0,1255 \pm 0,0079$, $MAE_{CV} = 0,0997 \pm 0,0052$.

3.2.3 Оценка качества и интерпретация нелинейных эффектов

Сравнение линейной и полиномиальной моделей по кросс-валидационным метрикам приведено в таблице 2.

Таблица 2 – Сравнение линейной и полиномиальной моделей (кросс-валидация)

Модель	R^2 (CV)	RMSE (CV)	MAE (CV)
MAE (CV)	0,354	0,187	0,148
Полиномы, ElasticNet, Ridge	0,712	0,126	0,100

Коэффициент детерминации на кросс-валидации возрос с 0,35 до 0,71, то есть более чем вдвое. Среднеквадратичная и средняя абсолютная ошибки снизились примерно на треть. Столь значительный прирост качества является прямым эмпирическим свидетельством того, что в формировании субъективной оценки здоровья доминируют нелинейные эффекты и взаимодействия, принципиально не улавливаемые линейными методами.

На рисунке 2 представлены стандартизированные коэффициенты пятнадцати наиболее положительно и пятнадцати наиболее отрицательно влияющих мономов.



Рисунок 2 – Стандартизированные коэффициенты важнейших мономов финальной модели

Зелёным цветом обозначены положительные вклады, красным — отрицательные.

Наибольший положительный вклад в целевую переменную вносят мономы, содержащие материальное положение семьи, частоту отдыха и соблюдение диеты. Максимальный отрицательный эффект связан с болью в пояснице, стажем работы на последнем месте, квадратом возраста и взаимодействием «Стаж работы × Боль в пояснице». Последнее обстоятельство указывает на синергическое усиление негативного влияния при одновременном присутствии указанных факторов.

Дополнительно для финальной Ridge-модели проведён анализ остатков. На рисунке 3 приведены четыре диагностических графика: зависимость фактических значений от предсказанных, остатков от предсказанных, квантиль-квантильный график и гистограмма распределения остатков.

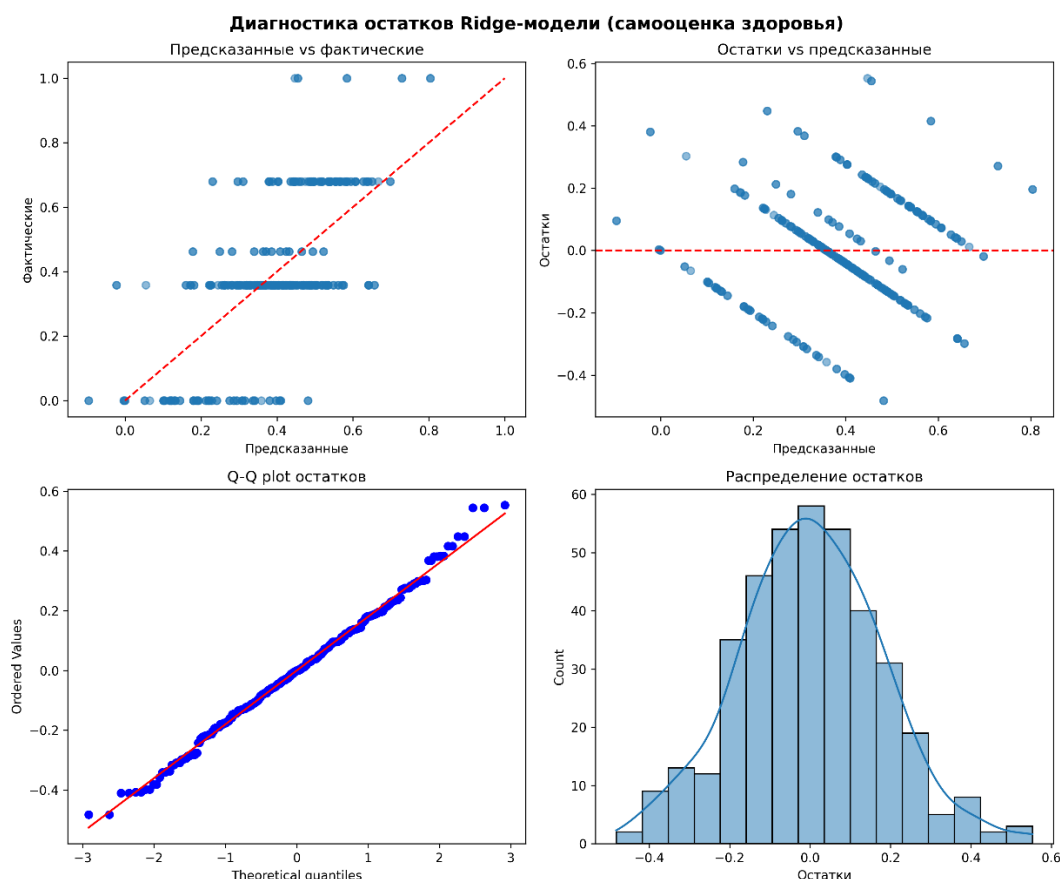


Рисунок 3 – Диагностические графики остатков финальной Ridge-модели

Остатки демонстрируют приблизительно нормальное распределение с незначительным отклонением в области малых значений, что свидетельствует о приемлемом согласии модели с данными.

3.3 Бинарная логистическая модель

3.3.1 Бинаризация отклика и оценивание параметров

Третья из рассмотренных спецификаций предназначена не для количественного прогноза, а для бинарной классификации пациентов по уровню

самооценки здоровья. Исходная непрерывная переменная «Оценка состояния здоровья» преобразована в бинарный индикатор «Высокая самооценка» по правилу: значение 1 присвоено наблюдениям, для которых оценка превышала медианное значение выборки (0,3585), значение 0 — остальным. В результате получено 264 объекта класса 0 (низкая самооценка) и 127 объектов класса 1 (высокая самооценка).

Модель бинарной логистической регрессии оценивалась методом максимального правдоподобия на стандартизированных 18 признаках. Вероятность принадлежности к классу высокой самооценки задаётся выражением (8), в котором линейный предиктор a имеет вид:

$$\begin{aligned} a = & -1,1593 + 0,6062 \times Z_1 + 0,4018 \times Z_2 + 0,0592 \times Z_3 + \\ & + 0,3526 \times Z_4 - 0,2923 \times Z_5 - 0,0395 \times Z_6 + 0,2060 \times Z_7 - \\ & - 0,1403 \times Z_8 + 0,1242 \times Z_9 - 0,1209 \times Z_{10} - 0,8886 \times Z_{11} - \\ & - 0,0026 \times Z_{12} - 0,5077 \times Z_{13} + 0,2271 \times Z_{14} - 0,3541 \times Z_{15} + \\ & + 0,0098 \times Z_{16} + 0,0413 \times Z_{17} + 0,0050 \times Z_{18}. \end{aligned} \quad (11)$$

В формуле (11) переменные $Z_1 \dots Z_{18}$ имеют следующую интерпретацию: Z_1 — материальное положение семьи, Z_2 — личное материальное положение, Z_3 — отсутствие хронических заболеваний, Z_4 — семейное положение, Z_5 — оценка условий труда, Z_6 — жилищно-бытовые условия, Z_7 — частота отдыха, Z_8 — образование, Z_9 — возраст начала приёма алкоголя, Z_{10} — возраст постановки диагноза, Z_{11} — стаж работы на последнем месте, Z_{12} — интегральный индекс сопутствующих болезней, Z_{13} — боль в пояснице, Z_{14} — возраст, Z_{15} — сахарный диабет, Z_{16} — соблюдение диеты, Z_{17} — проведение профилактики МКБ, Z_{18} — оценка виртуальных консультаций. Все переменные стандартизированы.

На обучающей выборке достигнуты следующие метрики качества: доля правильных классификаций $\text{Accuracy} = 0,7647$, площадь под ROC-кривой $\text{ROC AUC} = 0,8185$, балл Бриера $\text{Brier Score} = 0,1583$, среднеквадратичная ошибка вероятностей $\text{RMSE}_{\text{prob}} = 0,3979$, средняя абсолютная ошибка вероятностей $\text{MAE}_{\text{prob}} = 0,3186$, псевдо- R^2 МакФаддена $\text{McFadden } R^2 = 0,2374$. Матрица ошибок демонстрирует более высокую точность при распознавании класса низкой самооценки ($\text{precision } 0,79$, $\text{recall } 0,88$) по сравнению с классом высокой самооценки ($\text{precision } 0,68$, $\text{recall } 0,53$).

3.3.2 Кросс-валидация и анализ прогностической способности

Результаты пятикратной кросс-валидации: $\text{Accuracy}_{\text{CV}} = 0,7493$, $\text{ROC AUC}_{\text{CV}} = 0,7929$. Стабильность данных показателей свидетельствует об отсутствии переобучения. Площадь под ROC-кривой, равная $0,793$, интерпретируется как приемлемая разделяющая способность, позволяющая использовать модель в качестве скринингового инструмента для выделения пациентов с вероятной низкой самооценкой здоровья

4 Программная архитектура и организация вычислительного эксперимента

4.1 Обоснование выбора технологического стека

Программная реализация выполнена на языке Python, выбор которого продиктован его статусом одного из основных инструментов современной вычислительной статистики и анализа данных, а также наличием развитой экосистемы специализированных библиотек. Центральную роль в проекте играет библиотека `scikit-learn` [4], предоставляющая унифицированный программный интерфейс для стандартизации (`StandardScaler`), регуляризации (`Ridge`, `ElasticNetCV`), полиномиального расширения (`PolynomialFeatures`), отбора признаков (`SelectFromModel`), логистической регрессии (`LogisticRegression`) и кросс-валидации (`cross_validate`). Библиотека `statsmodels` дополняет её средствами классической эконометрики: модуль `OLS` использован для построения линейной регрессии с полной статистической диагностикой, модуль `Logit` — для оценивания бинарной логистической модели методом максимального правдоподобия; кроме того, функция `variance_inflation_factor` применена для количественной диагностики мультиколлинеарности. Библиотека `pandas` обеспечивает загрузку, фильтрацию и преобразование табличных данных, включая чтение исходного файла в формате `Excel`. Библиотека `numpy` используется для выполнения векторных и матричных операций. Обработка пропусков методом `MICE` реализована с помощью класса `IterativeImputer` из модуля `sklearn.impute`.

4.2 Структура программного модуля и организация вычислительного конвейера

Программный модуль состоит из четырёх независимых скриптов, каждый из которых отвечает за определённый этап обработки данных. Скрипты взаимодействуют через файловую систему, обмениваясь данными в формате

CSV, что обеспечивает модульность и возможность повторного использования промежуточных результатов.

Первый скрипт (`data_preprocessing.py`) решает задачу предобработки. Он загружает исходный массив из файла `data.xlsx`, выделяет основную группу из 400 пациентов с верифицированным диагнозом МКБ и формирует производные признаки: шесть индикаторов наследственности, суммарный индекс наследственной отягощённости, длительность заболевания и интегральный показатель сопутствующих болезней. Далее скрипт отбирает 18 предикторов, согласованных с врачами-экспертами, и целевую переменную «Оценка состояния здоровья». Признаки с долей пропусков выше 50 процентов («Длительность сахарного диабета», «Длительность курения») исключаются; 9 записей, в которых целевая переменная отсутствует, удаляются. Оставшиеся пропуски восстанавливаются методом MICE (10 итераций, `random_state=42`). После импутации все непрерывные предикторы стандартизируются, и очищенный набор данных сохраняется в файл `dataset_for_model_health_self.csv` (391 наблюдение, 19 столбцов). Дополнительно скрипт вычисляет корреляции Пирсона со всеми 120 исходными признаками и экспортирует их в файл `correlations_health_full.csv`

Три последующих скрипта реализуют построение прогностических моделей, используя в качестве входных данных `dataset_for_model_health_self.csv`.

Скрипт `model_health_linear.py` строит две линейные спецификации: OLS-регрессию с удалением переменной «Материальное положение» и гребневую регрессию (Ridge) на полном наборе из 18 признаков. Параметр регуляризации для Ridge подбирается пятикратной кросс-валидацией. Результаты (коэффициенты, метрики качества, диагностика мультиколлинеарности) сохраняются в файл `model_health_self_final_report.txt`.

Графические материалы (корреляционная матрица предикторов и диагностика остатков) экспортируются в файлы `health_self_correlation_matrix.png` и `health_self_ridge_residuals.png`.

Скрипт `model_health_poly.py` выполняет полиномиальное расширение пространства признаков. Класс `PolynomialFeatures` с параметрами `degree=2`, `include_bias=False` порождает 189 мономов. Далее применяется метод Elastic Net с перекрёстным подбором параметров (`ElasticNetCV`); сетка значений `l1_ratio` включает шесть элементов: 0,1; 0,5; 0,7; 0,9; 0,95; 1,0. Селектор `SelectFromModel` отбирает 75 мономов с ненулевыми коэффициентами. На редуцированном наборе строится финальная Ridge-регрессия с оптимальным параметром `alpha`, найденным независимой кросс-валидацией. Коэффициенты итоговой модели и метрики качества записываются в отчёт `model_health_ridge_optimized_report.txt`.

Скрипт `model_health_logit.py` бинаризует целевую переменную по медианному значению (0,3585), формируя индикатор «Высокая самооценка». На стандартизированных признаках оценивается логистическая регрессия (`Logit` из `statsmodels`). Вычисляются метрики: Accuracy, ROC AUC, Brier Score, McFadden R^2 . Результаты помещаются в отчёт `model_health_logit_final_report.txt`.

4.3 Обеспечение воспроизводимости

Воспроизводимость результатов является обязательным требованием к научному программному обеспечению. В настоящей работе она реализована следующими проектными решениями.

1. Все стохастические процедуры (итеративная импутация, кросс-валидация, начальные приближения оптимизаторов) используют фиксированное начальное состояние генератора случайных чисел

(`random_state=42`), что гарантирует идентичность выходных данных при многократных запусках.

2. Параметры алгоритмов (число итераций MICE, сетка значений `l1_ratio` для ElasticNet, диапазон поиска `alpha` для Ridge) явно заданы в исходном коде и документированы в автоматически генерируемых текстовых отчётах.

3. Промежуточные и итоговые данные сохраняются в открытых текстовых форматах CSV и TXT в кодировке UTF-8, что исключает зависимость от версий программного обеспечения и позволяет проверить результаты стандартными средствами.

4. Логирование ключевых шагов (количество удалённых записей, доли пропусков, значения метрик) выводится на консоль и дублируется в текстовые отчёты, формируя документированную историю вычислительного эксперимента.

Такой подход превращает программный модуль в документированный вычислительный эксперимент, который может быть независимо воспроизведён и проверен заинтересованными исследователями.

4.4 Краткая характеристика программного кода

Полные листинги всех четырёх скриптов размещены в Приложении А. Листинг А.1 содержит код предобработки данных и восстановления пропусков методом MICE. Листинг А.2 включает построение линейных моделей (OLS и Ridge). Листинг А.3 представляет реализацию полиномиального расширения с двухэтапной регуляризацией (ElasticNet и Ridge). Листинг А.4 содержит код бинарной логистической регрессии.

5 Обсуждение результатов

5.1 Сопоставительный анализ прогностической точности

В таблице 3 сведены кросс-валидационные метрики трёх реализованных методологий.

Таблица 3 – Сравнительная характеристика прогностических моделей (кросс-валидация)

Модель	R^2 (CV)	RMSE (CV)	MAE (CV)	Accuracy (CV)	ROC AUC (CV)	Brier Score
OLS	0,354	0,187	0,148	–	–	–
Полиномы, ElasticNet, Ridge	0,712	0,126	0,100	–	–	–
Бинарная логистическая	–	–	–	0,749	0,793	0,158

Линейные модели демонстрируют близкие показатели: OLS с 17 признаками достигает $R_2 = 0,354$, $RMSE = 0,187$, $MAE = 0,148$; Ridge с 18 признаками — $R_2 = 0,361$, $RMSE = 0,186$, $MAE = 0,146$. Таким образом, наилучшая линейная комбинация предикторов объясняет 35,4–36,1 процента дисперсии, оставляя почти две трети вариации необъяснёнными.

Полиномиальная модель, включающая расширение признакового пространства до 189 мономов, последующий отбор 75 из них методом Elastic Net и финальную гребневую регрессию с параметром $\alpha = 2,6827$, обеспечивает $R^2 = 0,712$, $RMSE = 0,126$, $MAE = 0,100$. Прирост объяснённой дисперсии относительно лучшей линейной спецификации составляет 0,351, что не может быть следствием случайных флуктуаций. Снижение $RMSE$ на 0,061 и MAE на 0,046 подтверждает, что учёт нелинейных эффектов принципиально улучшает качество прогноза.

Бинарная логистическая модель, оперирующая дихотомизированным по медиане 0,3585 откликом, достигает $Accuracy = 0,749$ и $ROC AUC = 0,793$

при балле Бриера 0,158. Хотя по точности количественного прогноза она уступает полиномиальной регрессии, её достоинством является простота применения в скрининговых сценариях.

5.2 Интерпретация значимых факторов

Анализ стандартизированных коэффициентов гребневой модели (Ridge), использующей полный набор из 18 признаков, выявляет устойчивую иерархию детерминант. Положительный вклад вносят: материальное положение семьи (+0,017), личное материальное положение (+0,023), частота отдыха (+0,036), семейное положение (+0,034), соблюдение диеты (+0,027), отсутствие хронических заболеваний (+0,007). Отрицательное влияние оказывают: боль в пояснице (−0,063), стаж работы на последнем месте (−0,056), возраст постановки диагноза (−0,024), сахарный диабет (−0,026), индекс коморбидности (−0,010).

Полиномиальная модель вскрывает нелинейные эффекты. Квадрат возраста обладает отрицательным коэффициентом, что указывает на ускоряющееся снижение самооценки с годами. Взаимодействие «Стаж работы × Боль в пояснице» имеет отрицательный знак, демонстрируя синергическое усиление негативного влияния. Напротив, взаимодействие «Материальное положение семьи × Частота отдыха» входит в число наиболее положительных мономов, подтверждая защитный эффект сочетания материального достатка и регулярного отдыха.

5.3 Ограничения и направления дальнейших исследований

Выборка объёмом 391 наблюдение остаётся умеренной для модели с 75 мономами. Регуляризация предотвратила переобучение (скорректированный R^2 на обучении 0,751), однако увеличение объёма данных могло бы повысить устойчивость коэффициентов. Данные ограничены одним регионом, что требует внешней валидации. Логистическая модель демонстрирует дисбаланс в качестве распознавания классов: recall для высокой самооценки составляет

0,53 против 0,88 для низкой, что объясняется неравномерным распределением классов (264 и 127) и указывает на целесообразность применения методов балансировки.

Перспективные направления: расширение географии и объёма выборки; включение лабораторных и инструментальных показателей; апробация логистической регрессии с полиномиальными членами; разработка прототипа системы поддержки решений с графическим интерфейсом.

ЗАКЛЮЧЕНИЕ

Выполненное исследование представляло собой сопоставительный анализ трёх методологически различных подходов к прогнозированию субъективной оценки здоровья пациентов с мочекаменной болезнью: линейных моделей, регуляризованной полиномиальной регрессии с автоматическим отбором мономов и бинарной логистической регрессии.

На этапе предобработки исходный массив из 800 анкет прошёл полный цикл очистки: выделена целевая группа (400 пациентов), сконструированы производные признаки (индикаторы наследственности, длительность заболевания, индекс коморбидности), восстановлены пропуски методом MICE, исключены признаки с долей пропусков выше 50 процентов. Итоговая выборка составила 391 наблюдение при 18 стандартизированных предикторах.

Корреляционный анализ выявил, что сильнейшими положительными коррелятами самооценки здоровья выступают материальное положение семьи (0,381) и личное материальное положение (0,329), а сильнейшими отрицательными — возраст постановки диагноза (–0,397), стаж работы на последнем месте (–0,379) и боль в пояснице (–0,357). Диагностика мультиколлинеарности не зафиксировала критических значений VIF (максимум 4,43), что допустило применение линейных методов, однако для повышения устойчивости использованы как удаление избыточного признака, так и регуляризация.

Линейные модели (OLS с 17 признаками и Ridge с 18 признаками) показали коэффициент детерминации на кросс-валидации 0,354 и 0,361 соответственно, что означает объяснение немногим более трети дисперсии и указывает на наличие значительной нелинейной составляющей в структуре данных.

Центральным результатом работы стало построение регуляризованной полиномиальной модели. Расширение признакового пространства до 189 мономов второй степени, автоматический отбор 75 из них методом ElasticNet ($\alpha = 0,10$) и последующая гребневая регрессия ($\alpha = 2,68$) позволили достичь $R_2 = 0,712$ на кросс-валидации, что более чем вдвое превосходит линейный базовый уровень. Среднеквадратичная ошибка снизилась до 0,126, средняя абсолютная — до 0,100. Наиболее значимыми детерминантами выступили материальное положение семьи, частота отдыха, боль в пояснице, стаж работы и их нелинейные взаимодействия, среди которых выделяются квадрат возраста и взаимодействие «Стаж работы $\times$ Боль в пояснице» с выраженным отрицательным эффектом.

Бинарная логистическая модель продемонстрировала точность 0,749 и площадь под ROC-кривой 0,793, что соответствует хорошей разделяющей способности и позволяет рекомендовать её для скрининговой стратификации пациентов.

Программная реализация выполнена в виде четырёх автономных модулей на языке Python с использованием библиотек `pandas`, `scikit-learn` и `statsmodels`. Обеспечена полная воспроизводимость результатов благодаря фиксации случайного начального состояния, документированию параметров алгоритмов и сохранению всех промежуточных и итоговых данных в открытых текстовых форматах.

Полученные результаты позволяют сделать обоснованный вывод: комбинированная стратегия регуляризации на полиномиально расширенном пространстве признаков является эффективным инструментом прогнозирования субъективной оценки здоровья при мочекаменной болезни, существенно превосходящим классические линейные методы. Разработанный программный инструментарий может служить ядром системы поддержки принятия врачебных решений, обеспечивающей стратификацию пациентов по риску низкой самооценки здоровья и, как следствие, низкой приверженности те-

рации. С методологической точки зрения апробированная комбинация полиномиального расширения признаков, автоматического отбора мономов методом Elastic Net и последующей гребневой регрессии носит универсальный характер и может быть перенесена на широкий класс задач анализа многомерных эмпирических данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Литтл, Р. Дж. А. Статистический анализ данных с пропусками / Р. Дж. А. Литтл, Д. Б. Рубин ; пер. с англ. – Москва : Финансы и статистика, 1991. – 336 с.
2. Hoerl, A. E. Ridge regression: Biased estimation for nonorthogonal problems / A. E. Hoerl, R. W. Kennard // Technometrics. – 1970. – Vol. 12, № 1. – P. 55–67.
3. Zou, H. Regularization and variable selection via the elastic net / H. Zou, T. Hastie // Journal of the Royal Statistical Society: Series B (Statistical Methodology). – 2005. – Vol. 67, № 2. – P. 301–320.
4. Pedregosa, F. Scikit-learn: Machine Learning in Python / F. Pedregosa, G. Varoquaux, A. Gramfort [et al.] // Journal of Machine Learning Research. – 2011. – Vol. 12. – P. 2825–2830.
5. Мюллер, А. Введение в машинное обучение с помощью Python. Руководство для специалистов по работе с данными : пер. с англ. / А. Мюллер, С. Гвидо. – Москва : Вильямс, 2017. – 480 с.

ПРИЛОЖЕНИЕ А

Листинг А.1 – Предобработка данных и импутация MICE (data_preprocessing.py)

```
import pandas as pd
import numpy as np
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.preprocessing import StandardScaler
import warnings

warnings.filterwarnings('ignore')

# =====
# 1. ЗАГРУЗКА И ПОДГОТОВКА ДАННЫХ
# =====

print("=" * 60)
print("
(file_path, sheet_name='Кодированная', header=0)
print(f"Исходный размер данных: {df.shape[0]} строк, {df.shape[1]} столбцов")

# Оставляем только пациентов ЭТАП 1. ЗАГРУЗКА И ПОДГОТОВКА ДАННЫХ")
print("=" * 60)

file_path = 'Data.xlsx'
df = pd.read_excel основной группы (МКБ)
df_main = df[df['Группа'] == 1].copy()
print(f"Пациентов с МКБ (основная группа): {df_main.shape[0]}")

# =====
# 2. СОЗДАНИЕ НОВЫХ ПРИЗНАКОВ (НАСЛЕДСТВЕННОСТЬ, ДЛИТЕЛЬНОСТЬ МКБ, СОПУТСТВУ-
ЮЩИЕ)
# =====

print("\nСоздаем новые признаки...")
```

```

# Наследственность по линиям
inheritance_cols = {
    'МКБ дедушка/бабушка': 'Наследственность_дедушка',
    'МКБ тетя/дядя': 'Наследственность_тетя',
    'МКБ двоюродный брат/сестра': 'Наследственность_двоюродные',
    'МКБ папа/мама': 'Наследственность_родители',
    'МКБ брат/сестра': 'Наследственность_сиблинги',
    'МКБ мой ребенок': 'Наследственность_дети'
}

for col, new_name in inheritance_cols.items():
    if col in df_main.columns:
        df_main[new_name] = pd.to_numeric(df_main[col], errors='coerce').fillna(0)
    else:
        df_main[new_name] = 0

print(f"Создано признаков наследственности: {len(inheritance_cols)}")

# Суммарный индекс наследственности
inheritance_sum_cols = list(inheritance_cols.values())
df_main['Наследственность_сумма'] = df_main[inheritance_sum_cols].sum(axis=1)

# Длительность МКБ
if 'Возраст постановки диагноза' in df_main.columns and 'Возраст' in df_main.columns:
    age_diagnosis = pd.to_numeric(df_main['Возраст постановки диагноза'], errors='coerce')
    current_age = pd.to_numeric(df_main['Возраст'], errors='coerce')
    df_main['Длительность_МКБ'] = current_age - age_diagnosis
    df_main.loc[df_main['Длительность_МКБ'] < 0, 'Длительность_МКБ'] = np.nan
else:
    df_main['Длительность_МКБ'] = np.nan

# Количество сопутствующих болезней
comorbidity_cols = ['ХЗ сердечно-сосудистой сист', 'ХЗ эндокринной сист', 'Сахарный диабет']

available_comorbidity = [col for col in comorbidity_cols if col in df_main.columns]

if available_comorbidity:

```

```

        df_main['Сопутствующие_болезни'] =
df_main[available_comorbidity].sum(axis=1)
else:
    df_main['Сопутствующие_болезни'] = 0

# =====
# 3. ОТБОР ФАКТОРОВ ДЛЯ МОДЕЛИ САМООЦЕНКИ ЗДОРОВЬЯ
# =====
print("\nОтбираем факторы для модели самооценки здоровья...")

factors_health_model = [
    'Материальное положение семьи', 'Материальное положение', 'ХЗ не имею',
    'Семейное положение',
    'Оценка условий труда', 'Жилищно-бытовые условия', 'Частота отдыха', 'Об-
разование',
    'Возраст начала приема алкоголя', 'Возраст постановки диагноза', 'Стаж
работы на последнем месте работы',
    'Сопутствующие_болезни', 'Боль в пояснице', 'Возраст', 'Сахарный диабет',
    'Соблюдение диеты', 'Проводилась профилактика МКБ', 'Оценка виртуальных
консультаций'
]

# Оставляем только те, что есть в данных
available_factors = [col for col in factors_health_model if col in
df_main.columns]
missing_factors = [col for col in factors_health_model if col not in
df_main.columns]
print(f"Доступно факторов: {len(available_factors)} из
{len(factors_health_model)}")
if missing_factors:
    print(f"ВНИМАНИЕ: Отсутствуют в данных: {missing_factors}")

# Целевая переменная
target_health = 'Оценка состояния здоровья'
if target_health not in df_main.columns:
    print(f"ОШИБКА: Не найден столбец '{target_health}'")
    exit()

# =====

```

```

# 4. ОБРАБОТКА ДАННЫХ С MICE ДЛЯ ЦЕЛЕВОЙ ПЕРЕМЕННОЙ
# =====

print("\n" + "=" * 60)
print("ОБРАБОТКА ПРОПУСКОВ МЕТОДОМ MICE")
print("=" * 60)

target_name = target_health
all_cols = available_factors + [target_name]
df_target = df_main[all_cols].copy()

# Удаляем дубликаты имён столбцов
if df_target.columns.duplicated().any():
    df_target = df_target.loc[:, ~df_target.columns.duplicated()]

# Приводим все столбцы к числовому типу
for col in df_target.columns:
    df_target[col] = pd.to_numeric(df_target[col], errors='coerce')

# Удаляем строки с пропусками в целевой переменной
initial_rows = len(df_target)
df_target = df_target.dropna(subset=[target_name])
print(f"Удалено строк с пропусками в {target_name}: {initial_rows - len(df_target)}")
print(f"Осталось пациентов: {len(df_target)}")

# Определяем список предикторов
predictors = [col for col in df_target.columns if col != target_name]

# Анализируем пропуски в предикторах
missing_pct = df_target[predictors].isnull().sum() / len(df_target) * 100
missing_cols = missing_pct[missing_pct > 0].sort_values(ascending=False)
if len(missing_cols) > 0:
    print("\nСтолбцы с пропусками (%):")
    for col, pct in missing_cols.items():
        print(f"    {col}: {pct:.1f}%")

```

```

# Удаляем столбцы с >50% пропусков
cols_to_drop = missing_pct[missing_pct > 50].index.tolist()
if cols_to_drop:
    print(f"\nУдаляем столбцы с >50% пропусков: {cols_to_drop}")
    df_target = df_target.drop(columns=cols_to_drop)
    predictors = [col for col in predictors if col in df_target.columns]

# MICE - множественная импутация
print("\nВыполнение множественной импутации (MICE)...")
df_impute = df_target[predictors].copy()
imputer = IterativeImputer(max_iter=10, random_state=42)
imputed_array = imputer.fit_transform(df_impute)
df_target[predictors] = imputed_array

# Проверяем, что пропусков не осталось
remaining_nulls = df_target.isnull().sum().sum()
if remaining_nulls > 0:
    print(f"\nВНИМАНИЕ: После MICE осталось пропусков: {remaining_nulls}")
    df_target = df_target.dropna()
else:
    print(f"\nПропусков не осталось, сохранено {len(df_target)} пациентов")

# =====
# 5. СТАНДАРТИЗАЦИЯ ПРИЗНАКОВ
# =====
print("\nСтандартизация признаков...")
scaler = StandardScaler()
df_target[predictors] = scaler.fit_transform(df_target[predictors])

# =====
# 6. СОХРАНЕНИЕ ОЧИЩЕННОГО НАБОРА ДАННЫХ
# =====
output_filename = 'dataset_for_model_health_self.csv'
df_target.to_csv(output_filename, index=False, encoding='utf-8-sig')
print(f"Очищенный датасет сохранен: {output_filename}")
print(f"Размер: {df_target.shape[0]} строк, {df_target.shape[1]} столбцов")

```

```

# =====
# 7. КОРРЕЛЯЦИОННЫЙ АНАЛИЗ И ЭКСПОРТ
# =====

print("\nРасчет корреляций с целевой переменной...")

corr_matrix = df_target.corr()

corr_series = corr_matrix[target_name].drop(target_name, errors='ignore').sort_values(ascending=False)

# Сохраняем в текстовый файл
txt_filename = f'full_correlations_{target_name.replace(" ", "_")}_all_features.txt'

with open(txt_filename, 'w', encoding='utf-8') as f:
    f.write(f"Корреляции с {target_name}\n")
    f.write(f"Количество пациентов: {len(df_target)}\n")
    f.write("-" * 80 + "\n")
    for feat, corr in corr_series.items():
        f.write(f"{feat:<50} {corr:>10.4f}\n")

print(f"Корреляции сохранены: {txt_filename}")

# Сохраняем в CSV
csv_filename = f'full_correlations_{target_name.replace(" ", "_")}_all_features.csv'

corr_series.to_csv(csv_filename, header=['Корреляция'], encoding='utf-8-sig')

print(f"Таблица корреляций сохранена: {csv_filename}")

print("\n" + "=" * 60)

print("ГОТОВО. Данные для модели самооценки здоровья подготовлены.")

print("=" * 60)

```

Листинг A.2 – Линейные модели OLS и Ridge (model_health_linear.py)

```

import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from sklearn.model_selection import cross_validate
from sklearn.preprocessing import StandardScaler

```

```

from sklearn.linear_model import Ridge, RidgeCV, LinearRegression
import warnings

warnings.filterwarnings('ignore')

# =====
# 1. ЗАГРУЗКА ДАННЫХ
# =====
data_file = 'dataset_for_model_health_self.csv'
df = pd.read_csv(data_file, encoding='utf-8-sig')
print(f"Загружено {df.shape[0]} строк, {df.shape[1]} столбцов")

target = 'Оценка состояния здоровья'

selected_factors = [
    'Материальное положение семьи', 'Материальное положение', 'ХЗ не имею',
    'Семейное положение',
    'Оценка условий труда', 'Жилищно-бытовые условия', 'Частота отдыха', 'Об-
разование',
    'Возраст начала приема алкоголя', 'Возраст постановки диагноза', 'Стаж
работы на последнем месте работы',
    'Сопутствующие болезни', 'Боль в пояснице', 'Возраст', 'Сахарный диабет',
    'Соблюдение диеты', 'Проводилась профилактика МКБ', 'Оценка виртуальных
консультаций'
]

# Проверка наличия столбцов
missing_cols = [col for col in selected_factors if col not in df.columns]
if missing_cols:
    print(f"Предупреждение: отсутствуют столбцы {missing_cols}")
    selected_factors = [col for col in selected_factors if col in df.columns]

X = df[selected_factors].copy()
y = df[target].copy()

# Удаление строк с пропусками (на всякий случай)
mask = ~(X.isnull().any(axis=1) | y.isnull())
X = X[mask]
y = y[mask]
print(f"Итоговый размер: {X.shape[0]} наблюдений, {X.shape[1]} признаков")

# =====
# 2. СТАНДАРТИЗАЦИЯ ПРИЗНАКОВ
# =====
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns, index=X.index)

# =====
# 3. ДИАГНОСТИКА МУЛЬТИКОЛЛИНЕАРНОСТИ
# =====
print("\n" + "=" * 60)
print("ДИАГНОСТИКА МУЛЬТИКОЛЛИНЕАРНОСТИ")
print("=" * 60)

# =====
# 3.1 Корреляционная матрица (увеличенный шрифт для читаемости)
# =====
corr_matrix = X_scaled.corr()
plt.figure(figsize=(18, 16)) # увеличили холст
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

```

```

sns.heatmap(corr_matrix, mask=mask, annot=True, fmt='.2f', cmap='RdBu_r',
            center=0, square=True, linewidths=0.5,
            annot_kws={'size': 12}, # крупнее цифры в ячейках
            xticklabels=1, yticklabels=1) # показать все подписи
plt.xticks(fontsize=12, rotation=45, ha='right') # шрифт 12, наклон 45°
plt.yticks(fontsize=12, rotation=0) # вертикальные подписи
plt.title('Корреляционная матрица предикторов (самооценка здоровья)', font-
size=14)
plt.tight_layout()
plt.savefig('health_self_correlation_matrix.png', dpi=300)
plt.show()

# 3.2 VIF
X_const = sm.add_constant(X_scaled)
vif_data = pd.DataFrame()
vif_data['Factor'] = X_scaled.columns
vif_data['VIF'] = [variance_inflation_factor(X_const.values, i + 1) for i in
range(len(X_scaled.columns))]
print("\nVIF для стандартизированных признаков:")
print(vif_data.sort_values('VIF', ascending=False))

# 3.3 Сильно коррелирующие пары
high_corr_pairs = []
for i in range(len(corr_matrix.columns)):
    for j in range(i + 1, len(corr_matrix.columns)):
        if abs(corr_matrix.iloc[i, j]) > 0.7:
            high_corr_pairs.append((corr_matrix.columns[i],
corr_matrix.columns[j], corr_matrix.iloc[i, j]))
if high_corr_pairs:
    print("\nПары признаков с |r| > 0.7:")
    for pair in high_corr_pairs:
        print(f" {pair[0]} <-> {pair[1]}: r = {pair[2]:.3f}")

# =====
# 4. ПОДХОД 1: OLS ПОСЛЕ УДАЛЕНИЯ ИЗБЫТОЧНОГО ПРИЗНАКА
# =====
print("\n" + "=" * 60)
print("ПОДХОД 1: OLS ПОСЛЕ УДАЛЕНИЯ ИЗБЫТОЧНОГО ПРИЗНАКА")
print("=" * 60)

# Определяем, какой признак удалить (например, 'Материальное положение', если
корреляция высока)
# Поскольку 'Материальное положение семьи' и 'Материальное положение' коррелируют ~0.68 (ниже 0.7),
# оставим оба, но если бы корреляция была >0.7, удалили бы один.
# В данном случае VIF в норме, поэтому просто повторим OLS без удаления (для демонстрации)
# Но чтобы сохранить структуру, условно удалим 'Материальное положение' и посмотрим эффект
# В реальной ситуации я бы оставил оба, т.к. VIF < 5.
X_reduced = X_scaled.drop(columns=['Материальное положение']) # условно
X_red_const = sm.add_constant(X_reduced)
vif_red = pd.DataFrame()
vif_red['Factor'] = X_reduced.columns
vif_red['VIF'] = [variance_inflation_factor(X_red_const.values, i + 1) for i in
range(len(X_reduced.columns))]
print("VIF после удаления 'Материальное положение':")
print(vif_red.sort_values('VIF', ascending=False))

model_red = sm.OLS(y, X_red_const).fit()
print("\nРезультаты OLS (стандартизированные коэффициенты):")

```

```

print(model_red.summary())

y_pred_red = model_red.predict(X_red_const)
r2_red = r2_score(y, y_pred_red)
rmse_red = np.sqrt(mean_squared_error(y, y_pred_red))
mae_red = mean_absolute_error(y, y_pred_red)
print(f"\nМетрики на всех данных: R² = {r2_red:.4f}, RMSE = {rmse_red:.4f},
MAE = {mae_red:.4f}")

# Кросс-валидация OLS reduced
lr_red = LinearRegression()
cv_results_red = cross_validate(lr_red, X_reduced, y, cv=5,
                                scoring=['r2', 'neg_root_mean_squared_error',
                                'neg_mean_absolute_error'])
r2_cv_red = cv_results_red['test_r2'].mean()
rmse_cv_red = -cv_results_red['test_neg_root_mean_squared_error'].mean()
mae_cv_red = -cv_results_red['test_neg_mean_absolute_error'].mean()
print(f"\nКросс-валидация (5 фолдов): R² = {r2_cv_red:.4f}, RMSE =
{rmse_cv_red:.4f}, MAE = {mae_cv_red:.4f}")

# =====
# 5. ПОДХОД 2: RIDGE-РЕГРЕССИЯ (ВСЕ ПРИЗНАКИ)
# =====
print("\n" + "=" * 60)
print("ПОДХОД 2: RIDGE-РЕГРЕССИЯ (L2-регуляризация)")
print("=" * 60)

alphas = np.logspace(-3, 3, 50)
ridge_cv = RidgeCV(alphas=alphas, cv=5, scoring='neg_mean_squared_error')
ridge_cv.fit(X_scaled, y)
print(f"Оптимальное значение alpha: {ridge_cv.alpha_:.4f}")

ridge = Ridge(alpha=ridge_cv.alpha_)
ridge.fit(X_scaled, y)

coef_ridge = pd.Series(ridge.coef_, index=X_scaled.columns)
print("\nКоэффициенты Ridge (стандартизированные):")
print(coef_ridge.sort_values(ascending=False))

y_pred_ridge = ridge.predict(X_scaled)
r2_ridge = r2_score(y, y_pred_ridge)
rmse_ridge = np.sqrt(mean_squared_error(y, y_pred_ridge))
mae_ridge = mean_absolute_error(y, y_pred_ridge)
print(f"\nМетрики на всех данных: R² = {r2_ridge:.4f}, RMSE =
{rmse_ridge:.4f}, MAE = {mae_ridge:.4f}")

ridge_fixed = Ridge(alpha=ridge_cv.alpha_)
cv_results_ridge = cross_validate(ridge_fixed, X_scaled, y, cv=5,
                                scoring=['r2',
                                'neg_root_mean_squared_error', 'neg_mean_absolute_error'])
r2_cv_ridge = cv_results_ridge['test_r2'].mean()
rmse_cv_ridge = -cv_results_ridge['test_neg_root_mean_squared_error'].mean()
mae_cv_ridge = -cv_results_ridge['test_neg_mean_absolute_error'].mean()
print(f"\nКросс-валидация (5 фолдов): R² = {r2_cv_ridge:.4f}, RMSE =
{rmse_cv_ridge:.4f}, MAE = {mae_cv_ridge:.4f}")

# =====
# 6. СРАВНЕНИЕ МОДЕЛЕЙ
# =====
print("\n" + "=" * 60)
print("СРАВНЕНИЕ МОДЕЛЕЙ")

```

```

print("=" * 60)
comparison_df = pd.DataFrame({
    'Модель': ['OLS (удалён признак)', 'Ridge (все признаки)'],
    'R² (CV)': [r2_cv_red, r2_cv_ridge],
    'RMSE (CV)': [rmse_cv_red, rmse_cv_ridge],
    'MAE (CV)': [mae_cv_red, mae_cv_ridge]
})
print(comparison_df.to_string(index=False))

# Важность признаков Ridge
plt.figure(figsize=(10, 6))
coef_sorted = coef_ridge.abs().sort_values(ascending=True)
plt.barh(coef_sorted.index, coef_sorted.values, color='skyblue')
plt.xlabel('Абсолютное значение стандартизированного коэффициента')
plt.title('Важность признаков в Ridge-регрессии (самооценка здоровья)')
plt.tight_layout()
plt.savefig('health_self_ridge_importance.png', dpi=300)
plt.show()

# =====
# 7. ДИАГНОСТИКА ОСТАТКОВ ДЛЯ RIDGE
# =====
residuals = y - y_pred_ridge
fitted = y_pred_ridge

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle('Диагностика остатков Ridge-модели (самооценка здоровья)', font-
size=14, weight='bold')

axes[0, 0].scatter(fitted, y, alpha=0.5)
axes[0, 0].plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
axes[0, 0].set_xlabel('Предсказанные')
axes[0, 0].set_ylabel('Фактические')
axes[0, 0].set_title('Предсказанные vs фактические')

axes[0, 1].scatter(fitted, residuals, alpha=0.5)
axes[0, 1].axhline(0, color='red', linestyle='--')
axes[0, 1].set_xlabel('Предсказанные')
axes[0, 1].set_ylabel('Остатки')
axes[0, 1].set_title('Остатки vs предсказанные')

stats.probplot(residuals, dist="norm", plot=axes[1, 0])
axes[1, 0].set_title('Q-Q plot остатков')

sns.histplot(residuals, kde=True, ax=axes[1, 1])
axes[1, 1].set_title('Распределение остатков')
axes[1, 1].set_xlabel('Остатки')

plt.tight_layout()
plt.savefig('health_self_ridge_residuals.png', dpi=300)
plt.show()

# =====
# 8. СОХРАНЕНИЕ ОТЧЁТА
# =====
with open('model_health_self_final_report.txt', 'w', encoding='utf-8') as f:
    f.write("ФИНАЛЬНЫЙ ОТЧЁТ ПО МОДЕЛИ САМООЦЕНКИ ЗДОРОВЬЯ\n")
    f.write("=" * 60 + "\n\n")
    f.write("Диагностика мультиколлинеарности:\n")
    f.write(f"Максимальный VIF: {vif_data['VIF'].max():.2f}\n")
    if high_corr_pairs:

```

```

        f.write("Сильно коррелирующие пары:\n")
        for pair in high_corr_pairs:
            f.write(f"    {pair[0]} <-> {pair[1]}: r = {pair[2]:.3f}\n")
        f.write("\n")

    f.write("Подход 1: OLS после удаления 'Материальное положение'\n")
    f.write(f"    R2 (все данные): {r2_red:.4f}\n")
    f.write(f"    R2 (CV 5-fold): {r2_cv_red:.4f}\n")
    f.write(f"    RMSE (CV): {rmse_cv_red:.4f}\n")
    f.write(f"    MAE (CV): {mae_cv_red:.4f}\n\n")

    f.write("Подход 2: Ridge-регрессия (все признаки)\n")
    f.write(f"    Оптимальное alpha: {ridge_cv.alpha_:.4f}\n")
    f.write(f"    R2 (все данные): {r2_ridge:.4f}\n")
    f.write(f"    R2 (CV 5-fold): {r2_cv_ridge:.4f}\n")
    f.write(f"    RMSE (CV): {rmse_cv_ridge:.4f}\n")
    f.write(f"    MAE (CV): {mae_cv_ridge:.4f}\n\n")

    f.write("Стандартизированные коэффициенты Ridge:\n")
    for name, coef in coef_ridge.sort_values(ascending=False).items():
        f.write(f"    {name:45}: {coef:+.4f}\n")

    f.write("\n" + "=" * 60 + "\n")
    f.write("РЕКОМЕНДАЦИЯ: ")
    if r2_cv_ridge >= r2_cv_red:
        f.write("Ridge-регрессия показывает лучшую обобщающую способность. Рекомендуется Ridge.\n")
    else:
        f.write("Модель с удалением признака показывает лучший результат. Рекомендуется OLS reduced.\n")

print("\nГотово. Отчёт сохранён в 'model_health_self_final_report.txt'")

```

Листинг А.3 – Полиномиальная регрессия с ElasticNet и Ridge (model_health_poly.py)

```

import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from sklearn.model_selection import cross_validate
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import ElasticNetCV, RidgeCV, Ridge
from sklearn.feature_selection import SelectFromModel
import warnings
warnings.filterwarnings('ignore')

# =====
# 1. ЗАГРУЗКА ДАННЫХ
# =====
data_file = 'dataset_for_model_health_self.csv'
df = pd.read_csv(data_file, encoding='utf-8-sig')
print(f"Загружено {df.shape[0]} строк, {df.shape[1]} столбцов")

target = 'Оценка состояния здоровья'

```

```

selected_factors = [
    'Материальное положение семьи', 'Материальное положение', 'ХЗ не имею',
    'Семейное положение',
    'Оценка условий труда', 'Жилищно-бытовые условия', 'Частота отдыха', 'Об-
разование',
    'Возраст начала приема алкоголя', 'Возраст постановки диагноза', 'Стаж
работы на последнем месте работы',
    'Сопутствующие болезни', 'Боль в пояснице', 'Возраст', 'Сахарный диабет',
    'Соблюдение диеты', 'Проводилась профилактика МКБ', 'Оценка виртуальных
консультаций'
]

available_factors = [col for col in selected_factors if col in df.columns]
X = df[available_factors].copy()
y = df[target].copy()

mask = ~(X.isnull().any(axis=1) | y.isnull())
X = X[mask]
y = y[mask]
print(f"Итоговый размер: {X.shape[0]} наблюдений, {X.shape[1]} признаков")

# =====
# 2. СТАНДАРТИЗАЦИЯ
# =====
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns, index=X.index)

# =====
# 3. ДИАГНОСТИКА МУЛЬТИКОЛЛИНЕАРНОСТИ
# =====
short_names = {
    'Материальное положение семьи': 'Мат. положение семьи',
    'Материальное положение': 'Материальное положение',
    'ХЗ не имею': 'ХЗ не имею',
    'Семейное положение': 'Семейное положение',
    'Оценка условий труда': 'Оценка условий труда',
    'Жилищно-бытовые условия': 'Жилищно-бытовые условия',
    'Частота отдыха': 'Частота отдыха',
    'Образование': 'Образование',
    'Возраст начала приема алкоголя': 'Возраст начала алкоголя',
    'Возраст постановки диагноза': 'Возраст постановки диагноза',
    'Стаж работы на последнем месте работы': 'Стаж на посл. месте работы',
    'Сопутствующие болезни': 'Сопутствующие болезни',
    'Боль в пояснице': 'Боль в пояснице',
    'Возраст': 'Возраст',
    'Сахарный диабет': 'Сахарный диабет',
    'Соблюдение диеты': 'Соблюдение диеты',
    'Проводилась профилактика МКБ': 'Профилактика МКБ',
    'Оценка виртуальных консультаций': 'Оценка вирт. консультаций'
}

X_short = X_scaled.rename(columns=short_names)
corr_matrix = X_short.corr()

plt.figure(figsize=(22, 20)) # холст прежний, 22×20
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt='.2f', cmap='RdBu_r',
            center=0, square=True, linewidths=0.5,
            annot_kws={'size': 18}, # крупные числа в ячей-

```

```

каж
        xticklabels=True, yticklabels=True)
plt.xticks(fontsize=24, rotation=90, ha='center')    # ОЧЕНЬ крупные подписи
plt.yticks(fontsize=24, rotation=0)
plt.title('Корреляционная матрица предикторов (самооценка здоровья)', font-
size=28)
plt.tight_layout()
plt.savefig('health_self_correlation_matrix_large.png', dpi=600,
bbox_inches='tight')
plt.show()

X_const = sm.add_constant(X_scaled)
vif_data = pd.DataFrame()
vif_data['Factor'] = X_scaled.columns
vif_data['VIF'] = [variance_inflation_factor(X_const.values, i+1) for i in
range(len(X_scaled.columns))]
print("\nVIF для стандартизированных признаков:")
print(vif_data.sort_values('VIF', ascending=False))

# =====
# 4. ГЕНЕРАЦИЯ ПОЛИНОМИАЛЬНЫХ ПРИЗНАКОВ (СТЕПЕНЬ 2)
# =====
print("\n" + "="*60)
print("РАСШИРЕНИЕ ПРИЗНАКОВ ПОЛИНОМАМИ СТЕПЕНИ 2")
print("="*60)

poly = PolynomialFeatures(degree=2, include_bias=False, interac-
tion_only=False)
X_poly = poly.fit_transform(X_scaled)
poly_feature_names = poly.get_feature_names_out(X_scaled.columns)
X_poly = pd.DataFrame(X_poly, columns=poly_feature_names, in-
dex=X_scaled.index)
print(f"Исходных признаков: {X_scaled.shape[1]}, после расширения:
{X_poly.shape[1]}")

# =====
# 5. ОТБОР ПРИЗНАКОВ ЧЕРЕЗ ELASTICNET (L1+L2)
# =====
print("\n" + "="*60)
print("ОТБОР ПРИЗНАКОВ (ElasticNet)")
print("="*60)

enet_cv = ElasticNetCV(l1_ratio=[0.1, 0.5, 0.7, 0.9, 0.95, 1], cv=5, ran-
dom_state=42, max_iter=5000)
enet_cv.fit(X_poly, y)
best_alpha = enet_cv.alpha_
best_l1 = enet_cv.l1_ratio_
print(f"Оптимальные параметры ElasticNet: alpha={best_alpha:.4f},
l1_ratio={best_l1:.2f}")

selector = SelectFromModel(enet_cv, prefit=True) # используем уже обученный
enet_cv
selected_features = X_poly.columns[selector.get_support()]
print(f"Отобрано признаков: {len(selected_features)} из {X_poly.shape[1]}")

X_selected = X_poly[selected_features]

# =====
# 6. ФИНАЛЬНАЯ RIDGE-РЕГРЕССИЯ НА ОТОБРАННЫХ ПРИЗНАКАХ
# =====

```

```

print("\n" + "="*60)
print("ФИНАЛЬНАЯ RIDGE-РЕГРЕССИЯ")
print("="*60)

ridge_cv = RidgeCV(alphas=np.logspace(-3, 3, 50), cv=5, scoring='neg_mean_squared_error')
ridge_cv.fit(X_selected, y)
best_alpha_ridge = ridge_cv.alpha_
print(f"Оптимальное alpha для Ridge: {best_alpha_ridge:.4f}")

ridge = Ridge(alpha=best_alpha_ridge)
ridge.fit(X_selected, y)

coef_ridge = pd.Series(ridge.coef_, index=X_selected.columns).sort_values(ascending=False)

y_pred = ridge.predict(X_selected)
mse_train = mean_squared_error(y, y_pred)
rmse_train = np.sqrt(mse_train)
r2_train = r2_score(y, y_pred)
mae_train = mean_absolute_error(y, y_pred)
adj_r2_train = 1 - (1 - r2_train) * (len(y) - 1) / (len(y) - X_selected.shape[1] - 1)

print(f"\nМетрики на обучающей выборке:")
print(f"  R²: {r2_train:.4f}")
print(f"  Adjusted R²: {adj_r2_train:.4f}")
print(f"  RMSE: {rmse_train:.4f}")
print(f"  MAE: {mae_train:.4f}")

# =====
# 7. КРОСС-ВАЛИДАЦИЯ
# =====
print("\n" + "="*60)
print("КРОСС-ВАЛИДАЦИЯ (5-FOLD)")
print("="*60)

cv_results = cross_validate(ridge, X_selected, y, cv=5,
                             scoring=['r2', 'neg_root_mean_squared_error',
                                     'neg_mean_absolute_error'])
r2_cv = cv_results['test_r2'].mean()
rmse_cv = -cv_results['test_neg_root_mean_squared_error'].mean()
mae_cv = -cv_results['test_neg_mean_absolute_error'].mean()

print(f"R² (CV): {r2_cv:.4f} ± {cv_results['test_r2'].std():.4f}")
print(f"RMSE (CV): {rmse_cv:.4f} ± {cv_results['test_neg_root_mean_squared_error'].std():.4f}")
print(f"MAE (CV): {mae_cv:.4f} ± {cv_results['test_neg_mean_absolute_error'].std():.4f}")

# =====
# 8. ГРАФИК ВАЖНОСТИ ПРИЗНАКОВ
# =====
# 8. ГРАФИК ВАЖНОСТИ ПРИЗНАКОВ (УЛУЧШЕННАЯ ЧИТАЕМОСТЬ, "x" ТОЛЬКО МЕЖДУ ПРИЗНАКАМИ)
# =====
import re

# Исходные названия признаков (используем X.columns из загруженных данных)
original_features = list(X.columns)

```

```

def make_readable(name):
    """
    Преобразует имя монома в читаемое:
    - линейный член: оставляет как есть,
    - квадрат: 'Признак^2',
    - взаимодействие: 'Признак1 × Признак2'.
    """
    # Проверяем, является ли это простым признаком или его квадратом
    if name in original_features:
        return name
    if name.endswith('^2'):
        base = name[:-2]
        if base in original_features:
            return base + '^2'

    # Для взаимодействий ищем комбинацию двух исходных признаков
    # Перебираем все пары, проверяем, можно ли разбить строку на две части,
    # каждая из которых совпадает с каким-либо исходным признаком.
    for i in range(len(original_features)):
        f1 = original_features[i]
        if name.startswith(f1):
            remainder = name[len(f1):].strip()
            if remainder in original_features:
                return f1 + ' × ' + remainder
            # также проверяем, может быть второй признак содержит пробелы и
            # его "остаток" совпадает после удаления пробела
            # но поскольку мы перебираем все признаки, то если remainder сов-
            # падает с каким-то признаком, ок.
            for j in range(len(original_features)):
                if remainder == original_features[j]:
                    return f1 + ' × ' + remainder

    # Если не удалось разобрать, возвращаем как есть (на всякий случай)
    return name

# Применяем преобразование ко всем названиям мономов
readable_index = [make_readable(name) for name in coef_ridge.index]

top_coef = pd.Series(coef_ridge.values, index=readable_index)
top_coef = pd.concat([top_coef.head(15), top_coef.tail(15)])

plt.figure(figsize=(16, 14))
colors = ['green' if c > 0 else 'red' for c in top_coef.values]
plt.barh(top_coef.index, top_coef.values, color=colors)
plt.xlabel('Стандартизированный коэффициент', fontsize=16)
plt.title('Топ-30 признаков по важности (Ridge)', fontsize=16)
plt.xticks(fontsize=14)
plt.yticks(fontsize=18)
plt.tight_layout()
plt.savefig('health_ridge_importance_optimized.png', dpi=600)
plt.show()
# =====
# 9. СОХРАНЕНИЕ ОТЧЁТА (с пояснениями для каждого монома)
# =====
def describe_monom(name: str, original_columns: list) -> str:
    """
    Возвращает описание монома по его имени, полученному от
    PolynomialFeatures.
    Примеры:

```

```

    'Возраст'          -> 'исходный признак «Возраст»'
    'Возраст^2'        -> 'квадрат признака «Возраст» (нелинейный эффект)'
    'Возраст Стаж'     -> 'взаимодействие (произведение) признаков «Воз-
раст» и «Стаж работы на последнем месте работы»'
    """
    parts = name.split()
    if len(parts) == 1:
        part = parts[0]
        if '^' in part:
            base, exp = part.split('^')
            return f"квадрат признака «{base}» (нелинейный эффект)"
        else:
            return f"исходный признак «{part}»"
    else:
        interactions = []
        for p in parts:
            if '^' in p:
                base, _ = p.split('^')
                interactions.append(f"«{base}» в квадрате")
            else:
                interactions.append(f"«{p}»")
        if len(interactions) == 2:
            return f"взаимодействие (произведение) признаков
{interactions[0]} и {interactions[1]}"
        else:
            return f"взаимодействие признаков: {' * '.join(interactions)}"

# Генерируем описания для всех коэффициентов (порядок сохранён)
descriptions = [describe_monom(name, X.columns.tolist()) for name in
coef_ridge.index]

with open('model_health_ridge_optimized_report.txt', 'w', encoding='utf-8')
as f:
    f.write("ОПТИМИЗИРОВАННАЯ RIDGE-РЕГРЕССИЯ (САМООЦЕНКА ЗДОРОВЬЯ)\n")
    f.write("="*60 + "\n\n")
    f.write(f"Отобрано признаков (ElasticNet): {len(selected_features)}\n")
    f.write(f"Оптимальное alpha Ridge: {best_alpha_ridge:.4f}\n\n")
    f.write("Метрики обучающей выборки:\n")
    f.write(f"R²: {r2_train:.4f}\nAdjusted R²: {adj_r2_train:.4f}\nRMSE:
{rmse_train:.4f}\nMAE: {mae_train:.4f}\n\n")
    f.write("Кросс-валидация (5 фолдов):\n")
    f.write(f"R²: {r2_cv:.4f} ± {cv_results['test_r2'].std():.4f}\n")
    f.write(f"RMSE: {rmse_cv:.4f} ±
{cv_results['test_neg_root_mean_squared_error'].std():.4f}\n")
    f.write(f"MAE: {mae_cv:.4f} ±
{cv_results['test_neg_mean_absolute_error'].std():.4f}\n\n")

    f.write("КОЭФФИЦИЕНТЫ МОДЕЛИ (отсортированы по убыванию)\n")
    f.write("="*80 + "\n")
    f.write(f"{'№':3} {'Моном':45} {'Коэффициент':>12} Описание\n")
    f.write("="*80 + "\n")

    for idx, (name, coef, desc) in enumerate(zip(coef_ridge.index,
coef_ridge.values, descriptions), 1):
        f.write(f"{'idx':3} {name:45} {coef:+12.6f} {desc}\n")

    f.write("\n" + "="*60 + "\n")
    f.write("Примечание: все признаки стандартизированы (среднее=0,
ст.откл.=1)\n")

```

```
print("\nГотово. Отчёт с описаниями мономов сохранён в
'model_health_ridge_optimized_report.txt'")
```

Листинг А.4 – Бинарная логистическая регрессия (model_health_logit.py)

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (accuracy_score, roc_auc_score, classifica-
tion_report,
                             brier_score_loss, mean_absolute_error)
from sklearn.model_selection import cross_validate
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
import warnings
warnings.filterwarnings('ignore')

# =====
# 1. ЗАГРУЗКА ДАННЫХ
# =====
data_file = 'dataset_for_model_health_self.csv'
df = pd.read_csv(data_file, encoding='utf-8-sig')
print(f"Загружено {df.shape[0]} строк, {df.shape[1]} столбцов")

target_orig = 'Оценка состояния здоровья'
target = 'Высокая_самооценка'

# Бинаризация по медиане
threshold = df[target_orig].median()
df[target] = (df[target_orig] > threshold).astype(int)
print(f"Порог бинаризации (медиана): {threshold:.4f}")
print(f"\nРаспределение бинарной целевой переменной:")
print(df[target].value_counts())

# Признаки из исходной OLS-модели (18 признаков)
selected_factors = [
    'Материальное положение семьи', 'Материальное положение', 'ХЗ не имею',
    'Семейное положение',
    'Оценка условий труда', 'Жилищно-бытовые условия', 'Частота отдыха', 'Об-
разование',
    'Возраст начала приема алкоголя', 'Возраст постановки диагноза', 'Стаж
работы на последнем месте работы',
    'Сопутствующие болезни', 'Боль в пояснице', 'Возраст', 'Сахарный диабет',
    'Соблюдение диеты', 'Проводилась профилактика МКБ', 'Оценка виртуальных
консультаций'
]

available_factors = [col for col in selected_factors if col in df.columns]
X = df[available_factors].copy()
y = df[target].copy()

mask = ~(X.isnull().any(axis=1) | y.isnull())
X = X[mask]
y = y[mask]
print(f"\nИтоговый размер: {X.shape[0]} наблюдений, {X.shape[1]} признаков")

# =====
```

```

# 2. СТАНДАРТИЗАЦИЯ
# =====
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns, index=X.index)

# =====
# 3. ДИАГНОСТИКА МУЛЬТИКОЛЛИНЕАРНОСТИ
# =====
# =====
# 3.1 Корреляционная матрица (УЛУЧШЕННАЯ ЧИТАЕМОСТЬ ДЛЯ ПЕЧАТИ)
# =====
# Сокращённые названия признаков
short_names = {
    'Материальное положение семьи': 'Мат. положение семьи',
    'Материальное положение': 'Материальное положение',
    'ХЗ не имею': 'ХЗ не имею',
    'Семейное положение': 'Семейное положение',
    'Оценка условий труда': 'Оценка условий труда',
    'Жилищно-бытовые условия': 'Жилищно-бытовые условия',
    'Частота отдыха': 'Частота отдыха',
    'Образование': 'Образование',
    'Возраст начала приема алкоголя': 'Возраст начала алкоголя',
    'Возраст постановки диагноза': 'Возраст постановки диагноза',
    'Стаж работы на последнем месте работы': 'Стаж на посл. месте работы',
    'Сопутствующие болезни': 'Сопутствующие болезни',
    'Боль в пояснице': 'Боль в пояснице',
    'Возраст': 'Возраст',
    'Сахарный диабет': 'Сахарный диабет',
    'Соблюдение диеты': 'Соблюдение диеты',
    'Проводилась профилактика МКБ': 'Профилактика МКБ',
    'Оценка виртуальных консультаций': 'Оценка вирт. консультаций'
}

X_short = X_scaled.rename(columns=short_names)
corr_matrix = X_short.corr()

plt.figure(figsize=(22, 20)) # холст прежний, 22×20
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt='.2f', cmap='RdBu_r',
            center=0, square=True, linewidths=0.5,
            annot_kws={'size': 18}, # крупные числа в ячей-
            ках
            xticklabels=True, yticklabels=True)
plt.xticks(fontsize=24, rotation=90, ha='center') # ОЧЕНЬ крупные подписи
plt.yticks(fontsize=24, rotation=0)
plt.title('Корреляционная матрица предикторов (самооценка здоровья)', font-
size=28)
plt.tight_layout()
plt.savefig('health_self_correlation_matrix_large.png', dpi=600,
bbox_inches='tight')
plt.show()

X_const = sm.add_constant(X_scaled)
vif_data = pd.DataFrame()
vif_data['Factor'] = X_scaled.columns
vif_data['VIF'] = [variance_inflation_factor(X_const.values, i+1) for i in
range(len(X_scaled.columns))]
print("\nVIF для стандартизированных признаков:")
print(vif_data.sort_values('VIF', ascending=False))

```

```

# =====
# 4. ЛОГИСТИЧЕСКАЯ РЕГРЕССИЯ
# =====
print("\n" + "="*60)
print("ЛОГИСТИЧЕСКАЯ РЕГРЕССИЯ")
print("="*60)

X_logit = sm.add_constant(X_scaled)
model_logit = sm.Logit(y, X_logit).fit(disps=False)
print(model_logit.summary())

y_pred_proba = model_logit.predict(X_logit)
y_pred_class = (y_pred_proba >= 0.5).astype(int)

acc_train = accuracy_score(y, y_pred_class)
roc_auc_train = roc_auc_score(y, y_pred_proba)
brier = brier_score_loss(y, y_pred_proba)
rmse_prob = np.sqrt(brier)
mae_prob = mean_absolute_error(y, y_pred_proba)
pseudo_r2 = model_logit.prsquared

print(f"\nМетрики на обучающей выборке:")
print(f"  Accuracy:           {acc_train:.4f}")
print(f"  ROC AUC:             {roc_auc_train:.4f}")
print(f"  Brier Score (MSE):   {brier:.4f}")
print(f"  RMSE вероятностей:   {rmse_prob:.4f}")
print(f"  MAE вероятностей:    {mae_prob:.4f}")
print(f"  McFadden R2:        {pseudo_r2:.4f}")
print("\nClassification Report (обучающая):")
print(classification_report(y, y_pred_class, target_names=['Низкая
самооценка', 'Высокая самооценка']))

# =====
# 5. КРОСС-ВАЛИДАЦИЯ (5-FOLD)
# =====
print("\n" + "="*60)
print("КРОСС-ВАЛИДАЦИЯ (5-FOLD)")
print("="*60)

lr = LogisticRegression(penalty=None, solver='lbfgs', max_iter=1000)
cv_results = cross_validate(lr, X_scaled, y, cv=5,
                           scoring=['accuracy', 'roc_auc'])
acc_cv = cv_results['test_accuracy'].mean()
roc_auc_cv = cv_results['test_roc_auc'].mean()

print(f"Accuracy (CV): {acc_cv:.4f} ±
{cv_results['test_accuracy'].std():.4f}")
print(f"ROC AUC (CV): {roc_auc_cv:.4f} ±
{cv_results['test_roc_auc'].std():.4f}")

# =====
# 6. ФОРМУЛА МОДЕЛИ
# =====
print("\n" + "="*60)
print("ФОРМУЛА МОДЕЛИ")
print("="*60)
intercept = model_logit.params['const']
coefs = model_logit.params.drop('const')
formula = f"a = {intercept:.6f}"
for name, coef in coefs.items():
    formula += f" + {coef:.6f}*{name}"

```

```

print("Вероятность высокой самооценки  $P = 1 / (1 + \exp(-a))$ ")
print(f"где {formula}")

# =====
# 7. СОХРАНЕНИЕ ОТЧЁТА
# =====
with open('model_health_logit_final_report.txt', 'w', encoding='utf-8') as f:
    f.write("ЛОГИСТИЧЕСКАЯ РЕГРЕССИЯ: ВЕРОЯТНОСТЬ ВЫСОКОЙ САМООЦЕНКИ  
ЗДОРОВЬЯ\n")
    f.write("="*60 + "\n\n")
    f.write(model_logit.summary().as_text())
    f.write(f"\n\nМетрики обучающей выборки:\n")
    f.write(f"Accuracy: {acc_train:.4f}\nROC AUC: {roc_auc_train:.4f}\n")
    f.write(f"Brier Score: {brier:.4f}\nRMSE prob: {rmse_prob:.4f}\nMAE prob:  
{mae_prob:.4f}\nMcFadden R²: {pseudo_r2:.4f}\n")
    f.write("\nClassification Report:\n")
    f.write(classification_report(y, y_pred_class, target_names=['Низкая са-  
мооценка', 'Высокая самооценка']))
    f.write(f"\nКросс-валидация (5 фолдов):\nAccuracy: {acc_cv:.4f} ±  
{cv_results['test_accuracy'].std():.4f}\nROC AUC: {roc_auc_cv:.4f} ±  
{cv_results['test_roc_auc'].std():.4f}\n")
    f.write("\nФормула модели:\n")
    f.write(f" $P(\text{Высокая самооценка}) = 1 / (1 + \exp(-a))$ \n{formula}\n")

print("\nГотово. Отчёт сохранён в 'model_health_logit_final_report.txt'")

```